The BX3 Framework: Architectural Accountability for Recursive Multi-Agent Systems

Jeremy Beebe[1,1]

[1]**Bxthre3 Inc, Monte Vista, Colorado, USA**

Abstract

We present the **BX3 Framework** (Behavior, Execution, and eXchange Architecture) — a novel architectural framework for recursive multi-agent AI systems that enforces human accountability as a compulsory structural property, not a policy aspiration. The framework organizes every node into three immutable functional layers — **Purpose** (intent and final authority), **Bounds** (bounded reasoning within provisioned operating ranges), and **Fact** (deterministic enforcement and forensic auditability) — and introduces five named architectural pillars that collectively constitute an **upstream accountability guarantee**: that BX3 systems fail upward into human consciousness, never downward into autonomous chaos.

The five pillars are: (1) **Loop Isolation**, which prevents state circularity in recursive execution graphs; (2) **Recursive Spawning**, which maintains immutable parent-child accountability chains as nodes spawn subordinate agents; (3) **Spatial Firewall**, which enforces execution domain separation between spawned sub-agents; (4) **Sandbox Gate**, which provides controlled inter-domain communication channels with mandatory authorization verification; and (5) **Bailout Protocol**, which compels every unresolved exception state to propagate upward to a named human principal, bypassing all intermediate machine actors, with immutable forensic attribution at every step.

We provide formal specifications for each pillar, establish the correctness properties they jointly guarantee, and describe the complete BX3 three-layer implementation model. Evaluated against the Tiered Agentic Oversight (TAO) model, the BX3 Framework provides stronger accountability guarantees: TAO permits absorption at intermediate machine tiers; the BX3 Framework forbids it architecturally. We further characterize the framework's properties under adversarial conditions and identify the directed research agenda required to extend these guarantees to high-stakes production deployments.

Keywords: multi-agent systems, accountability, human-in-the-loop, fail-safe design, recursive AI, architectural frameworks, BX3, Bailout Protocol

[1]Corresponding author. Email: jeremy@bxthre3.com

1 Introduction

1.1 The Autonomous Drift Problem in Recursive Multi-Agent Systems

When a multi-agent system decomposes a directive into sub-tasks and spawns subordinate agents to execute those sub-tasks, each spawning generation introduces compounding divergence from the human principal's original intent. This phenomenon is called **autonomous drift**: the progressive, generation-over-generation deviation of agent behavior from the principal's defined objectives, operating bounds, and success criteria, occurring without architectural constraint on the direction or magnitude of deviation.

Autonomous drift is not a governance concern. It is an architectural property of systems that distribute decision-making authority across machine actors without enforcing structural accountability for the delegation chain. In a recursive spawning topology — where agents spawn sub-agents that spawn further sub-agents — drift compounds multiplicatively across spawning generations. A single maladapted node at depth three spawns children whose operating ranges are defined relative to a misaligned parent; by depth seven, the executing frontier operates in a context so far removed from the principal's intent that human review of individual decisions is structurally meaningless.

The drift problem is distinct from the reliability problem. A system may be reliable — agents execute their assigned tasks without failure — while still exhibiting drift, if the tasks to which they are assigned have drifted from the principal's actual intent. Reliability within a misaligned scope does not satisfy the accountability requirement. The BX3 Framework treats drift as the primary threat model, not agent failure.

The consequences extend beyond misaligned outcomes. Drift severs the accountability chain between the human principal and the executing nodes through three compounding failure modes.

1. **Contingent oversight failure.** When a system handles exceptions through policy-level constraints, human involvement becomes a runtime option determined by the agent network itself — precisely when that network is least qualified to assess its own competence. An agent in an exception state is, by definition, in a condition its designers did not fully anticipate. Delegating to that same agent the decision of whether its condition requires human intervention vests escalation authority in the least qualified actor in the chain.

2. **Structural unauditability.** Systems without an architectural fallback can absorb, reframe, or suppress exception conditions internally. Decision classes intended to require human awareness become invisible to external auditors because the system has sufficient agency to reclassify constraint violations as optimization opportunities [**?**]. The accountability chain between the human principal and the outcome is not merely weakened — it is structurally severed.

3. **Drift amplification.** Each successive decision made under unacknowledged exception produces a system state further from the principal's intent. In recursive spawning deployments, drift amplification compounds across spawning generations at a rate that environmental resistance or resource exhaustion may not intercept before consequential action has already been taken.

These three failure modes are not eliminated by incremental improvement to error handling or oversight practices. They are architectural properties of systems that treat accountability as a policy constraint. Policy constraints operate above the execution layer; they can be reinterpreted or deferred by agents with sufficient agency. Architectural fallbacks operate within the execution layer: they are triggered deterministically by conditions the agent cannot control and cannot route around.

1.2 Why Existing Approaches Are Insufficient

The dominant approaches to bounding agent spawning depth are Time-To-Live (TTL) limits and depth counters. Neither is structurally sufficient.

TTL limits constrain the operational lifetime of a spawned agent but do not constrain the operating range or objectives assigned to it. An agent with unlimited scope but a 30-second TTL can produce arbitrarily consequential outcomes before expiration. TTL addresses persistence, not authority. It says nothing about what the agent is permitted to do — only how long it may do it.

Depth counters constrain the number of spawning generations but do not constrain what each spawned generation is authorized to do. A depth-limited system can spawn agents at each permitted depth with unbounded operating ranges, and the accountability chain degrades with each generation regardless of whether the total depth is bounded at five or fifty. Depth limit addresses *chain length*, not *chain integrity*.

Both approaches share a fundamental flaw: they constrain *how far* spawning can go without constraining *what each spawned node is authorized to do*. The drift problem is not a depth problem — it is an authorization problem. The relevant quantity is not the number of generations but the fidelity of the delegation chain: whether each spawning edge correctly transmits the principal's intent and operating bounds, and whether that fidelity degrades or is corrupted as the chain extends.

A structurally sound approach to the drift problem requires three constraints applied jointly at every spawning edge:

1. The parent's operating range must be narrowed — not replicated — at the child. The child's scope must be a proper subset of the parent's, ensuring that spawning never expands authority.

2. The parent pointer must be immutable for the child's entire operational lifetime, preventing retroactive corruption of the delegation chain.

3. Every spawning event must be recorded as an immutable entry in the Forensic Ledger, making the delegation chain fully auditable in both directions.

These three constraints jointly define the **immutable parent pointer chain** — the structural property that makes drift structurally impossible. The chain is immutable because: each child is created with a fixed parent reference that no machine actor can modify; the child's operating range is constrained to be a strict subset of its parent's by the *Bounds* layer's pre-commit enforcement; and the complete spawn history is recorded in the Forensic Ledger, making reconstruction of the chain mandatory and auditable.

Drift becomes structurally impossible when the delegation chain preserves fidelity at every spawning edge. A corrupted or misaligned child cannot spawn further children with broader authority than its own; the chain of proper subset constraints propagates the principal's intent downward, and any violation triggers the Bailout Protocol before the child can act outside its provisioned range.

1.3 Formal Definition of Autonomous Drift

Let $T = (V, E)$ be a spawn tree where each vertex $v \in V$ is a node and each edge $(u, v) \in E$ represents a spawn event. Let $R(v)$ denote the operating range of node v, $I(v)$ the intent specification provisioned at node creation, and $d(v)$ the depth of node v from the root (human principal). Let $D_{\max}$ be the deployment depth bound.

Drift at node v is defined as the divergence $\delta(v)$ between the executing behavior of v and the intent $I(\text{root})$ of the human principal, measured as a function of the deviation between $R(v)$ and the subspace of $R(\text{root})$ that v was authorized to occupy given its depth and position in the spawn tree:

$$\delta(v) \;=\; \frac{|R(v) \,\Delta\, \mathcal{A}(v)|}{|R(\text{root})|}$$

where $\mathcal{A}(v)$ is the authorized operating subspace for v — the intersection of $R(\text{root})$ with all scope constraints inherited along the parent-pointer chain from the root to v — and Δ denotes symmetric set difference. Drift is defined as zero when $\mathcal{A}(v) = R(v)$; it is undefined when $\mathcal{A}(v) = \emptyset$.

Drift amplification across a spawning edge (u, v) is defined as the ratio of drift magnitudes:

$$\eta(u \to v) \;=\; \frac{\delta(v)}{\delta(u)}$$

A system is **drift-stable** at depth k if for all spawning edges in the subtree of depth k, $\eta(u \to v) \leq 1$. A system exhibits **drift amplification** if there exists at least one spawning edge where $\eta(u \to v) > 1$.

The BX3 framework ensures drift-stability through two mechanisms applied at every spawning edge: scope inheritance ($R(c) \subset R(p)$) ensures that $\mathcal{A}(v)$ is always a proper subspace of $\mathcal{A}(\alpha(v))$, limiting the authorized operating range at each generation; and immutable parent pointers ensure that the chain of authorized subspaces cannot be retroactively modified to widen authority at any node.

Without these constraints, a single drift-amplifying spawning edge can compound across multiple generations, causing $\delta(v)$ to grow without bound within the bounded depth of the spawn tree. With these constraints, drift at any node is bounded by the accumulated proper subset relations along the chain — and because each subset is strict and finite, the total drift across any valid chain is finite.

1.4 Paper Roadmap

The remainder of this paper is organized as follows. Section 3 provides the background across three research traditions that motivate the framework. Section 4 develops the BX3 three-layer model as the architectural foundation. Section 9 introduces all five architectural pillars with full formal specifications, with Pillar 2 (Recursive Spawning) receiving the definitive treatment. Section 10 provides the dedicated treatment of the Bailout Protocol, including its formal state machine specification, its interaction with the Forensic Ledger, and its behavior under recursive spawning conditions. Section 11 establishes the correctness properties the framework jointly guarantees. Section 12 evaluates the framework against related work. Section 13 characterizes structural limitations and the directed research agenda. Section 14 concludes.

2 The BX3 Framework: Scope, Purpose, and Contribution

The BX3 Framework proposes that accountable, safe, and auditable multi-agent AI systems can be constructed through five architectural guarantees, each enforced at the structural level rather than the policy level. These guarantees are not heuristics or behavioral guidelines — they are structural requirements whose violation is architecturally impossible within the framework. The framework organizes every node into three immutable functional layers (*Purpose*, *Bounds*, *Fact*) and provides

five named architectural pillars that together constitute the full upstream accountability guarantee: that BX3 systems fail upward into human consciousness, never downward into autonomous chaos.

The five pillars are:

Pillar 1: Loop Isolation. No two nodes share mutable execution state without mediated communication through the *Fact* layer, preventing state corruption and ensuring that the provenance of every decision is traceable to its originating node. This isolation is the prerequisite for the Forensic Ledger's ability to record an accurate, complete attribution chain.

Pillar 2: Recursive Spawning with Immutable Parent Pointers. Every spawned node carries a cryptographically sealed reference to its parent $\alpha(n)$ established at node creation and immutable for the node's lifetime. The immutable parent chain is the propagation substrate for the Bailout Protocol — it ensures that exception states have a structurally fixed escalation path that no machine actor can redirect.

Pillar 3: Spatial Firewall. Nodes operating in distinct domains communicate exclusively through the *Fact* layer, preventing cross-domain state contamination and ensuring that exceptions in one domain cannot silently propagate to others through shared mutable state. The spatial firewall composes with the parent chain to provide a two-dimensional boundary: horizontal (execution isolation via Loop Isolation) and vertical (domain separation via Spatial Firewall).

Pillar 4: Sandbox Gate. Untrusted code executes in a bounded execution environment instrumented by the *Fact* layer with pre-commit hooks that enforce resource limits, action visibility, and the deterministic refusal of unmediated privileged operations. The Sandbox Gate is the enforced execution boundary that makes the *Bounds* layer's operating range enforcement architecturally operative — not a policy constraint but a structural gate.

Pillar 5: Bailout Protocol. Any unresolved exception condition in any node must compulsorily escalate to the node's designated human anchor $h(n)$ through the immutable parent chain, bypassing all intermediate machine actors. No machine actor may serve as a terminal resolution point for an unresolved exception; only $h(n)$ may issue binding resolution directives. The Bailout Protocol is the runtime expression of the upstream accountability guarantee.

These five pillars are not independent design choices. They form an interdependent system whose correctness properties require the simultaneous operation of all five. Loop Isolation (P1) and the immutable parent chain (P2) together provide the propagation substrate for the Bailout Protocol (P5); without P1 and P2, the protocol has no reliable path. The Spatial Firewall (P3) prevents cross-domain contamination that would make exception state attribution unreliable. The Sandbox Gate (P4) ensures that the Bounds Engine's operating range enforcement is not bypassed by privileged operations. The Bailout Protocol (P5) closes the accountability loop for all conditions that exceed the *Bounds* layer's provisioned scope.

The BX3 Framework's contribution is not any single pillar. It is the structured composition of all five as a unified accountability architecture, with formally specified correctness properties and a directed research agenda that addresses the gaps between current specification and deployment at scale in high-stakes environments.

2.1 Relationship to the Bailout Protocol Paper

The Bailout Protocol paper provides the exhaustive, self-contained specification of Pillar 5. It is a complete, independently readable treatment of the protocol's formal state machine, trigger condition, escalation algorithm, bypass mechanism, timeout handling, correctness properties (Safety, Liveness, Accountability), limitations, and future work. That specification is incorporated by reference into the BX3 Framework as Pillar 5 and is not repeated in full here.

This paper provides the complete treatment of the framework itself — its three-layer model,

all five pillars including the Bailout Protocol in summary form, the unified correctness properties across all pillars, related work situating the framework in the broader research landscape, and the limitations and research agenda. Where this paper references the Bailout Protocol's formal specification, it points to the corresponding section of the standalone paper. Where this paper describes the pillars in summary, the full specification is in the respective pillar papers.

The separation is intentional: the BX3 Framework paper is a *framework paper*, not a *single-pillar paper*. It is designed to be read as a coherent architectural treatment, with each pillar's full specification available in its own dedicated paper for readers who need the complete technical detail.

3 Background

The Recursive Spawning Protocol (RSP) draws from and contributes to five distinct research traditions: agent lifecycle management in multi-agent systems; accountability and provenance in distributed AI architectures; the structural distinction between fixed and mutable delegation chains; hierarchical task decomposition in AI; and the BX3 Framework as the three-layer accountability substrate on which immutable parent chains become architecturally feasible rather than merely policy-dependent. This section surveys each tradition in turn, establishes the specific structural gaps RSP is designed to fill, and demonstrates that these gaps cannot be closed by extensions of existing approaches.

3.1 Agent Lifecycle Management in Multi-Agent Systems

The problem of governing agent populations across their full lifecycle—from provisioning through decommissioning—has become a central research concern as autonomous agents move from single-task deployments into complex, multi-agent workflows. The Cloud Native Computing Foundation's 2026 cloud-native agentic standards document identifies *agent identity*, least-privilege access, and tenancy controls as foundational requirements for multi-agent systems operating in Kubernetes environments [**?**]. The standards specify that agents requiring access beyond a user's permission scope or persisting beyond session termination must carry distinct identities enabling authentication, authorization, and audit enforcement at both the identity and execution layers. This identity requirement extends to Just-in-Time (JIT) access provisioning, Attribute-Based Access Control (ABAC), and Policy-Based Access Control (PBAC) for flexible, secure policy definition [**?**].

Microsoft's Agent Governance Toolkit operationalizes these principles through Agent OS—a stateless policy engine intercepting agent actions at sub-millisecond latency—and Agent Mesh, which provides cryptographic decentralized identifiers (DIDs) with Ed25519 signatures, an Inter-Agent Trust Protocol (IATP), and dynamic trust scoring across behavioral tiers [**?**]. This work demonstrates that structural runtime enforcement of agent identity and trust boundaries is achievable at production scale; however, neither the CNCF standards nor Microsoft's toolkit addresses the structural immutability of the parent delegation chain that RSP requires.

The Agent Lifecycle Protocol (ALP) defines a formal state machine governing six lifecycle stages—Genesis, Fork, Migration, Retirement, Succession, and Decommission—with explicit transition rules, boundary hooks, and a genealogical registry [**?**]. ALP identifies *lineage tracking* and *accountability across generations* as core unsolved challenges, observing that a comprehensive genealogical registry for querying an agent's complete family and history is necessary for audit and compliance but absent from conventional runtimes. RSP directly addresses this gap: the immutable parent pointer $\alpha(n)$ is the foundational primitive for the genealogical registry ALP calls for, and the forensic ledger maintained by the Fact Layer provides the append-only chain of custody that

makes lineage verification runtime-computable rather than reconstructed post-hoc from fragmented logs.

The Human Delegation Protocol (HDP) introduces a JSON-based token binding a human principal to authorization scope and session, with each agent in a multi-hop chain appending signed hops to an append-only provenance record [**?**, **?**]. HDP demonstrates that multi-hop, append-only human provenance in agentic workflows is achievable and verifiable offline using only the issuer's public key. RSP adopts HDP's append-only chain principle and extends it structurally: while HDP achieves provenance through signed hop records, RSP achieves structural immutability of the parent pointer at the Fact Layer write protocol level, eliminating the possibility of retroactive chain modification regardless of the signing scheme employed. The core structural problem RSP addresses is *spawn chain opacity*: the inability to determine, at any point during or after execution, whether a given agent's authority traces to a human principal or to an autonomous predecessor that may have drifted from its authorized purpose. This opacity arises because parent pointers are mutable in conventional architectures—they can be redirected by administrative action, policy override, or runtime reconfiguration—making retroactive provenance reconstruction inherently unreliable.

3.2 Accountability and Provenance in Distributed Systems

The accountability problem in distributed systems—how to construct a tamper-evident record of which actor performed which action, when, and under what authorization—has been studied extensively in blockchain, database, and operating systems research. The W3C PROV data model formalizes provenance as entities, activities, and agents linked by relations of derivation, attribution, and responsibility [**?**]. Under PROV-DM, agents can be attributed to other agents, enabling a provenance graph encoding the full chain of responsibility. The critical insight PROV contributes is that the provenance graph is only as reliable as the immutability guarantees of its edges: if the graph is mutable, any attribution derived from it is conditional rather than definitive [**?**].

PROV-AGENT extends the W3C PROV standard with agent-centric entities, activities, and relations, integrating agent tools, prompt/response interactions, model invocations, and telemetry into a unified provenance graph [**?**]. PROV-AGENT demonstrates convergence between agentic workflow tracking and classical provenance standards, enabling semantic traversals of agent decision chains. However, PROV-AGENT's provenance graph is constructed from runtime observations and logged events; it does not enforce structural immutability of the parent pointer at provisioning time. The provenance record is accurate only to the extent that logged events are complete and unmodified—a property that RSP's Fact Layer write protocol makes structurally guaranteed rather than conventionally assumed.

The immutable ledger tradition in distributed systems provides the structural model. Certificate Transparency demonstrates that append-only logging with hash-chaining can produce tamper-evident records without centralized trust [**?**]. Bitcoin's blockchain extends this with consensus-based append-only ledgers demonstrating that sufficiently distributed consensus can produce globally agreed records with no trusted central authority [**?**]. The critical distinction RSP makes is between *runtime-verified accountability* and *retroactive reconstruction*: a forensic ledger records events after they occur and can be used to infer what happened, whereas an immutable delegation chain is verified structurally at every spawn event through the Fact Layer pre-commit hook, confirming the parent pointer's integrity and the scope subset relationship before any child agent is provisioned.

SentinelAgent introduces a Delegation Chain Calculus (DCC) with seven formal properties governing delegation chains in federal multi-agent AI systems, demonstrating that six of seven properties remain robust under adversarial stress while intent preservation degrades under sophisticated paraphrasing attacks [**?**]. DCC's formalization of scope-action fidelity and cascade damage bounds

provides a precise vocabulary for RSP's correctness properties: drift prevention, chain integrity, and termination. RSP provides the structural complement to DCC's analytical framework by enforcing DCC's required properties architecturally through the Fact Layer's write protocol, rather than relying on policy-level enforcement that can be bypassed by sufficiently privileged actors.

3.3 Fixed vs. Mutable Delegation Chains

The structural distinction between fixed and mutable delegation chains is the central design choice determining whether accountability is achievable in a multi-agent system. A mutable delegation chain allows the parent pointer $\alpha(n)$ to be rewritten after provisioning—by administrative action, policy update, or by a sufficiently privileged actor. A fixed (immutable) delegation chain establishes $\alpha(n)$ once at provisioning and enforces immutability through the Fact Layer write protocol, making it structurally impossible to redirect after the fact.

Mutable delegation chains are the historical norm. OAuth 2.1's token exchange model enables scoped, time-limited delegation but leaves the delegation graph itself mutable: authorized users can re-delegate, administrators can modify scope, and tokens can be revoked retroactively [**?**]. UCAN (User Controlled Authorization Networks) introduces self-sovereign authorization with a capability model where delegation chains are encoded as capability graphs verified transitively. UCAN's capability graph is mutable: capabilities can be delegated, transferred, and revoked, with revocation propagating through the graph. This is appropriate for human-controlled authorization networks where the principal remains in the loop, but is structurally insufficient for autonomous agent chains where the human principal may not be reachable at decision time.

Google DeepMind's *Intelligent AI Delegation* framework identifies recursive delegation risk as a critical failure mode in long agent chains, proposing that delegation should transfer not just work but also scoped authority, responsibility, and accountability, with monitoring and trust mechanisms that hold when environments change or agents fail [**?**]. The IETF draft on identity management for agentic AI identifies recursive delegation risk—agents creating sub-agents or delegating tasks introducing complex chains of accountability and control—as a critical future challenge, with preemptive authorization and careful scoping proposed as mitigations [**?**]. RSP provides the structural complement to these policy-level approaches: rather than relying on the correctness of individual authorization decisions, RSP enforces that every spawn event satisfies the scope subset constraint and depth bound as structural invariants, making the delegation chain's integrity independent of the correctness of any individual actor's judgment.

The fundamental problem with mutable delegation chains in multi-agent systems is *accountability laundering*: a drifted agent's chain is rewritten to appear authorized, retroactively obscuring the original provenance. RSP eliminates accountability laundering by making the parent pointer immutable regardless of the requestor's identity or privileges: no administrative override exists, and the immutability is enforced at the write protocol level before any modification is executed. This contrasts with ACL-based immutability in conventional systems, where administrative compromise leads directly to immutability violation. BX3's protocol-level enforcement has no administrative override path [**?**, 1].

3.4 Hierarchical Task Decomposition in AI

Hierarchical task decomposition—the decomposition of complex objectives into subtask hierarchies allocated to specialized agents—is the dominant paradigm for multi-agent orchestration in contemporary AI systems. The design space spans from classical planning formalisms to LLM-native orchestration frameworks.

Classical HTN (Hierarchical Task Network) planning decomposes a high-level task into a hierarchy of subtasks, where each decomposition is a method specifying how a compound task can be replaced by a partially ordered set of subtasks, terminating when all primitive tasks are executable by the system's primitive action set [**?**]. Simon's concept of bounded rationality provides the theoretical foundation: due to cognitive limits, agents cannot evaluate all possible courses of action at once and must decompose complex problems into manageable subproblems [**?**, **?**]. This insight applies directly to modern LLM-based agents: an agent with a complex goal must decompose it because the full goal space exceeds its representational and computational capacity.

Modern LLM-based agents have revived hierarchical task decomposition under the rubric of dynamic, runtime spawning. AgentOrchestra introduces a hierarchical multi-agent framework combining high-level planning with modular, specialized agents, where a central planner decomposes objectives into sub-goals and delegates to role-specific sub-agents, demonstrating that hierarchical decomposition improves task success rates (83.39% on GAIA) over flat-agent baselines [**?**]. HALO introduces a three-layer hierarchical framework—high-level planning agent, mid-level role-design agents, and low-level inference agents—using Monte Carlo Tree Search to explore agent action spaces, outperforming static role assignment by approximately 14.4% on average across benchmarks [**?**]. THREAD treats LLM generation as a dynamic multi-threaded process where a parent thread can spawn child threads to handle sub-tasks and pause until those children return concise results, enabling hierarchical task decomposition while managing context length [**?**]. ReAcTree extends this with hierarchical task trees where each node can dynamically expand into sub-goals based on environmental feedback, demonstrating that dynamic decomposition outperforms static task graphs in noisy environments [**?**].

What connects classical HTN and modern LLM-based decomposition is the spawn tree: a hierarchical structure in which each node represents a subtask and edges represent the decomposition relationship. In both traditions, the tree is typically mutable and implicit—decomposition decisions are not recorded as first-class authorization artifacts, and child nodes are not distinguished from parent nodes in terms of their accountability status. A classical HTN planner's subtasks are subroutine calls without independent accountability chains; an LLM agent's spawned sub-agents are independent processes, but the chain of authorization is implicit in the agent's context window rather than explicit in an immutable warrant structure. Dynamic hierarchical task decomposition in open-loop agentic systems has been identified as a vector for *autonomous drift*: the progressive expansion of agent behavior beyond authorized scope, precisely because decomposition is unconstrained [**?**]. RSP adopts the hierarchical decomposition paradigm but adds the accountability layer absent from both traditions: each node in the decomposition tree is a first-class accountability entity with an immutable parent pointer, a defined termination condition, and a cryptographically verifiable chain back to a human anchor. Decomposition is not merely a planning technique; it is an authorization event that creates a new accountable entity.

3.5 The BX3 Framework as Substrate for Immutable Parent Chains

The BX3 Framework provides the minimal sufficient three-layer accountability architecture on which the Recursive Spawning Protocol operates [**?**, **?**]. BX3 defines three functional layers—the *Purpose Layer*, the *Bounds Engine*, and the *Fact Layer*—each defined by the properties it must maintain rather than by the type of actor occupying it. The Purpose Layer carries intent, judgment, and final human accountability; the Bounds Engine carries constrained execution within a defined operating range; the Fact Layer carries deterministic enforcement and forensic auditability. RSP is the second named pillar of the BX3 Framework, complementing Loop Isolation, Spatial Firewall, Sandbox Gate, and the Bailout Protocol [**?**, 1].

The upstream accountability guarantee of the BX3 Framework states that when any node encounters a state it cannot resolve within its provisioned bounds, accountability escalates recursively upward through the system hierarchy, bypassing all machine actors, until it reaches a human anchor [**?**, **?**]. This guarantee is not a policy choice; it is a structural property of the three-layer architecture. The Fact Layer creates the immutable parent pointer at spawn time, issues the spawn warrant, and maintains the forensic ledger that records every spawn event. The Fact Layer's determinism is what makes the parent pointer architecturally immutable: once written, the pointer cannot be updated because the Fact Layer does not permit modifications to its authorization records. Any attempt to overwrite $\alpha(n)$ is rejected at the protocol level before execution.

The depth bound $D_{\max}$ is a Purpose-Layer-defined, Bounds-Engine-enforced, Fact-Layer-hardened constraint. The Purpose Layer defines the maximum permissible chain depth as part of the spawn authorization policy. The Bounds Engine evaluates each proposed spawn against this limit, returning `SPAWN_REFUSED` if the spawn would exceed it. The Fact Layer's pre-commit hook enforces this limit structurally: any spawn operation that would produce $|C| \geq D_{\max}$ is rejected before execution, regardless of the Bounds Engine's return. The constraint is thus enforced three times—by the layer that defines it, the layer that evaluates against it, and the layer that structurally blocks violations. Removing any layer causes the corresponding guarantee to collapse: without the Purpose Layer, there is no authoritative human anchor and no source of spawn authorization policy; without the Bounds Engine, depth limits become unenforceable conventions subject to override by operational urgency; without the Fact Layer, immutability of the parent pointer and tamper-evidence of the ledger are promises rather than constraints.

The five BX3 pillars collectively establish the conditions under which the upstream accountability guarantee applies: that BX3 systems fail upward into human consciousness, never downward into autonomous chaos. Loop Isolation, Recursive Spawning, Spatial Firewall, and Sandbox Gate address exception conditions anticipated at design time and resolvable within the machine-only execution graph. RSP specifically addresses the structural conditions that enable autonomous drift—the mutable parent pointer, the absent scope refinement constraint, and the missing depth bound enforcement—making the drift triad architecturally impossible through the Fact Layer's write protocol immutability, the Purpose Layer's spawn authorization gate, and the Bounds Engine's depth enforcement.

4 The BX3 Three-Layer Model

The BX3 three-layer model is the architectural substrate within which all five pillars of the framework operate. It is not a separate mechanism from the pillars — it is the structural foundation that makes the pillars possible and that makes their guarantees unconditional rather than contingent. This section specifies each layer's function, the interaction constraints between layers, the data structures maintained by each layer, and the invariants that the layers collectively enforce.

4.1 Layer Definitions

Every BX3 node is organized into three immutable functional layers, each with a distinct role in the accountability architecture and each with strictly delineated authority that cannot be exceeded or transferred.

The *Purpose* layer carries intent, judgment, and final authority. It is the layer at which the human principal's objectives are translated into operational directives and at which the operating range $R(n)$ for node n is defined. The *Purpose* layer receives all escalated exceptions and issues all binding resolution directives. It is the exclusive terminus for any exception state that exceeds

machine-actor provisioned authority. The layer is non-delegable: no machine actor possesses the judgment capacity to serve as a substitute terminus for exceptions that exceed its own operating range. This non-delegability is not a policy constraint – it is a structural property of the layer's function. The *Purpose* layer carries the principal's intent, and no agent operating under delegated authority can possess the principal's intent in sufficient completeness to substitute for it.

The *Bounds* layer carries bounded reasoning and constrained execution within $R(n)$. It is the layer at which the node evaluates whether proposed actions fall within its current authority by reference to the authorization ledger maintained by the *Fact* layer, executes resolution procedures when exception conditions are detected, and emits the `bailout_signal` when the exception exceeds its current scope. The *Bounds* layer acts autonomously within $R(n)$ but is architecturally compelled to escalate when it exits that range. The T_{bounds} timeout parameter is enforced at this layer, ensuring that bounded reasoning does not become unbounded deliberation.

The *Fact* layer carries deterministic enforcement and forensic auditability. It is the layer at which the authorization ledger is consulted, the state machine transition relations of all five pillars are enforced, and the append-only Forensic Ledger is maintained. The *Fact* layer is the fail-safe anchor of the framework: it is the only layer that may issue a definitive halt to autonomous action, and it does so by refusing to approve any proposed action that lacks a valid authorization entry. No machine actor may modify, delete, or suppress entries in the Forensic Ledger; the layer's function is to enforce the record rather than to interpret it.

The three layers are not merely a conceptual taxonomy – they are enforced architectural divisions with distinct execution semantics enforced by the *Fact* layer's pre-commit hook. A node in the *Bounds* layer cannot issue authorizations; a node in the *Purpose* layer cannot spawn sub-agents without a designated human anchor; a node in the *Fact* layer cannot define operating ranges. These constraints are not convention – they are structural invariants enforced at the pre-commit stage before any layer's code can execute.

4.2 The Authorization Ledger

The authorization ledger is the definitive record of all actions that the *Fact* layer has approved for execution. Every entry in the ledger is immutable once written: no machine actor, including the *Bounds* layer or any parent node, may modify or delete a ledger entry after it is committed. The ledger is maintained by the *Fact* layer and is consulted by the *Bounds* layer before any proposed action is dispatched to the execution engine.

Each ledger entry e is a 4-tuple:

$$e = (t,\ \text{node}(e),\ \text{action}(e),\ \text{scope}(e))$$

where t is the logical timestamp, $\text{node}(e)$ is the identifier of the node that requested the authorization, $\text{action}(e)$ is the action descriptor, and $\text{scope}(e)$ is the operating range parameter within which the action was evaluated. The scope field is critical: it records the $R(n)$ parameter under which the action was authorized, enabling retrospective determination of whether the action was taken within or outside the principal's stated intent at the time of authorization.

The ledger provides the foundation for the BX3 accountability model: every consequential action is attributable to a node and authorized within a defined scope, and the complete action history of any node is reconstructable from the ledger as an immutable chronological record. The ledger is not a conventional audit log – it is an append-only authorization substrate that the *Fact* layer enforces at the pre-commit stage.

4.3 The Forensic Ledger

The Forensic Ledger is the append-only record of all protocol events across all five pillars of the framework. Unlike the authorization ledger, which records only approved actions, the Forensic Ledger records every state transition, every trigger condition, every timeout event, every escalation, every pathway selection decision, and every human directive issued by an $h(n)$. The Forensic Ledger is maintained exclusively by the *Fact* layer; no machine actor may write to, modify, or suppress any entry.

Each Forensic Ledger entry f is a 6-tuple:

$$f = (\mathrm{id}(f),\ t_{\mathrm{logical}}(f),\ \mathrm{node}(f),\ \mathrm{event}(f),\ \mathrm{context}(f),\ \mathrm{hash}(f))$$

where $\mathrm{id}(f)$ is the unique identifier of the entry, $t_{\mathrm{logical}}(f)$ is the logical timestamp assigned by the *Fact* layer, $\mathrm{node}(f)$ is the originating node, $\mathrm{event}(f)$ is the event type, $\mathrm{context}(f)$ is the full diagnostic context associated with the event (the content of which depends on the event type), and $\mathrm{hash}(f)$ is the SHA-256 hash of the preceding entry, forming a cryptographically chained sequence from the genesis entry to the most recent. The cryptographic chaining ensures that no entry can be modified, deleted, or inserted without breaking the chain, a condition detectable by any auditor without access to the ledger's contents.

The Forensic Ledger serves two distinct functions that are often conflated in conventional audit systems. First, it is an *enforcement mechanism*: the *Fact* layer's pre-commit hooks for the Bailout Protocol, Sandbox Gate, and Spatial Firewall all query the Forensic Ledger to determine the current state of their respective state machines before committing any state change. The ledger is not merely a record – it is the authoritative source of truth for the runtime state of all five pillars. Second, it is an *evidence mechanism*: any human principal, regulator, or external auditor can reconstruct the complete exception and resolution history of any node from the ledger, including all diagnostic context, all pathway selections, and all human directives, with cryptographic assurance that the record has not been tampered with.

The Forensic Ledger is the architectural bridge between the BX3 Framework's runtime accountability properties and its retrospective auditability properties. The Bailout Protocol ensures that every unresolved exception reaches $h(n)$; the Forensic Ledger ensures that this fact can be demonstrated to any external auditor at any future point in time. Together they constitute *accountable escalation*: the guarantee that exceptions are not merely resolved but resolved in a way that is permanently attributable.

4.4 Layer Interaction Constraints

The three layers interact through strictly defined interfaces that enforce the separation of concerns essential to the framework's accountability properties. These interfaces are not conventions – they are structural constraints enforced by the *Fact* layer's pre-commit hook.

IC-1. The *Bounds* layer may request authorization for an action by submitting an authorization request to the *Fact* layer. The *Fact* layer consults the authorization ledger and returns approval or rejection. The *Bounds* layer may not bypass this request-response cycle for any consequential action. The *Purpose* layer does not participate in this cycle – it defines the scope, it does not execute within it.

IC-2. The *Purpose* layer receives exceptions from the *Fact* layer only when the *Bounds* layer has triggered the Bailout Protocol and the protocol has reached the *Purpose* layer via the escalation algorithm. The *Purpose* layer does not receive raw exception signals – it receives escalated exceptions with full diagnostic context appended by the propagation chain.

IC-3. The *Fact* layer writes to the Forensic Ledger for every state transition of every pillar's state machine and for every event that requires recording under the pillar's specification. The *Fact* layer does not filter, aggregate, or summarize entries – it records every event in full. The *Purpose* and *Bounds* layers have no write access to the Forensic Ledger.

IC-4. The *Purpose* layer issues binding resolution directives by committing an entry to the authorization ledger through the *Fact* layer. The directive is not effective until it is recorded in the ledger and verified by the *Fact* layer's pre-commit hook. No machine actor may issue, simulate, or substitute a resolution directive.

These interaction constraints collectively enforce the separation of concerns that makes the BX3 accountability architecture sound. The *Purpose* layer defines intent but does not execute within scope; the *Bounds* layer executes within scope but cannot authorize actions outside it; the *Fact* layer enforces both constraints and records both the actions and the exceptions. No layer can substitute for another, and no layer can perform its functions without the others.

4.5 Three-Layer Invariants

The following invariants hold for every BX3 node throughout its operational lifetime. They are established at node provisioning, maintained by the *Fact* layer's pre-commit hooks, and are inviolable by any machine actor operating within the framework's architecture.

Invariant 1 (Intent Preservation) *For every action a executed by node n, there exists a ledger entry e such that $\mathit{action}(e) = a$ and $\mathit{scope}(e) \subseteq R(n)$, where $R(n)$ is the operating range defined by the Purpose layer of n.*

Invariant 2 (Immutable Parent Pointer) *The parent pointer $\alpha(n)$ of every node n is established at node provisioning and is never modified during the node's operational lifetime. No machine actor, including the Bounds layer and any parent node, may modify $\alpha(n)$.*

Invariant 3 (Exclusive Human Terminus) *For every node n, the human anchor $h(n)$ is established at node provisioning and is never modified during the node's operational lifetime. No machine actor may serve as the terminus for an unresolved exception state at node n. The only permissible terminus is $h(n)$.*

Invariant 4 (Authorization Ledger Immutability) *Once an entry e is written to the authorization ledger, it is never modified or deleted. The ledger is append-only, and its integrity is enforced by the Fact layer's pre-commit hook.*

Invariant 5 (Forensic Ledger Completeness) *Every state transition, every trigger condition, every timeout event, every human directive, and every exception event in any pillar's state machine is recorded in the Forensic Ledger before any subsequent action is taken by any layer of node n.*

These five invariants are not independent design choices – they are mutually defining structural facts. Invariant 1 depends on Invariant 4 for its enforcement; Invariant 3 depends on Invariant 2 for the propagation path; Invariant 5 depends on Invariant 4 for the integrity of the recorded context. Together they constitute the structural foundation for the upstream accountability guarantee: every consequential action is within scope, every exception is attributable, and every unresolved exception reaches $h(n)$.

5 Pillar 1: Loop Isolation

5.1 Conceptual Motivation

Multi-agent systems composed from agents that observe each other's outputs, act on shared state, and recursively spawn sub-agents are structurally susceptible to feedback loops — cycles of mutual causation in which an agent's output influences its own input through a path that includes other agents. Feedback loops are not inherently harmful; they are the operational substrate of coordinated systems. They become dangerous when they are invisible — when the agents that participate in a cycle are unaware that they do, when the system's self-observation is distorted by the very activity it observes, or when a loop amplifies a perturbation into a runaway deviation before any human principal can intervene.

Existing multi-agent frameworks that permit inter-agent state observation suffer from what we term *loop entanglement*: each agent's observation function is a function of the global state, and the global state is a function of all agents' actions. In such architectures, any agent observing any part of the system is observing a quantity that its own actions have influenced — directly or through the chain of other agents' actions. The result is that the act of observing the system changes the system, and the system that is changed is the system being observed. In benign cases, this produces oscillatory behavior that stabilizes. In adversarial or novel conditions, it produces unbounded amplification: the system observes a deviation, acts to correct it, and the correction produces a larger deviation that the system again acts to correct.

Loop Isolation addresses this at the architectural level. Rather than relying on agents to recognize and break cycles — a judgment that requires global knowledge the agent cannot possess — the BX3 framework enforces isolation boundaries that prevent any cycle from spanning more than one loop. Each loop is a closed causal domain: agents within the loop may observe and act on each other, but the loop's boundary prevents its internal dynamics from propagating back into the external system as a causal force, and prevents external causal forces from entering the loop as unmediated inputs.

The upstream accountability guarantee depends critically on Loop Isolation. If loops could propagate unbounded deviations across the agent network, the system state reaching the human anchor would be an amplification product of multiple nested cycles rather than a direct record of the originating exception. The human principal would receive distorted context, and the accountability record would conflate the original failure with the loop dynamics that followed it.

5.2 Formal Definition

A BX3 loop L is a directed acyclic graph (DAG) of nodes $\mathcal{N}(L)$ such that the causality relation $C \subseteq \mathcal{N}(L) \times \mathcal{N}(L)$ induced by observation and action dependencies is acyclic. The causality relation is defined by the inter-agent observation graph: for any two nodes $n, m \in \mathcal{N}(L)$, $(n, m) \in C$ if and only if the output of n is an input to m through direct observation or action dependency.

The Loop Isolation invariant states that for every pair of nodes $n, m \in \mathcal{N}(L)$ that are causally linked through direct or transitive inter-agent observation, there exists no path from m back to n through the observation graph that does not pass through the loop's boundary interface. Formally:

$$\forall n, m \in \mathcal{N}(L) : (n \to^* m) \wedge (m \to^+ n) \implies (n, m) \text{ cross the boundary of } L \tag{1}$$

where $\to^*$ denotes the reflexive transitive closure of the causality relation and $\to^+$ denotes its transitive closure (excluding the reflexive case). This invariant is structural: it is enforced by the

Fact layer's pre-commit hook at loop provisioning and cannot be violated by any machine actor at runtime.

The boundary interface of loop L consists of the set of permitted cross-boundary causal transmissions — an explicitly enumerated set of channels through which information may cross the loop's causal perimeter. Every cross-boundary transmission is intercepted by the interface controller and validated against this enumeration before it is permitted to propagate.

5.3 State Machine Formalism

The Loop Isolation mechanism is formally modeled as a deterministic finite state machine embedded in the *Fact* layer of every loop-capable node:

$$\mathcal{L} \;=\; (Q_L,\; q_0^L,\; \Sigma_L,\; \rightarrow_L,\; Q_F^L)$$

where:

- $Q_L = \{\,\text{BOUNDARY_OPEN},\ \text{BOUNDARY_CLOSED},\ \text{LOOP_ISOLATED},\ \text{LOOP_TERMINATED}\,\}$ is the set of loop states;
- $q_0^L = \text{BOUNDARY_OPEN}$ is the initial state, entered when loop provisioning begins;
- Σ_L comprises boundary-establishment events e_{boundary}, loop-composition signals e_{compose}, violation events e_{violate}, isolation confirmation e_{isolated}, and human-authorized termination $e_{\text{terminate}}$;
- $\rightarrow_L \subseteq Q_L \times \Sigma_L \times Q_L$ is the deterministic transition relation (Table 1);
- $Q_F^L = \{\text{LOOP_TERMINATED}\}$ is the terminal accepting state.

Table 1: Loop Isolation state transition relation $\rightarrow_L \subseteq Q_L \times \Sigma_L \times Q_L$. Rows are current state; columns are input events. Blank cells indicate no defined transition.

	Input event				
Current state	e_{boundary}	e_{compose}	e_{violate}	e_{isolated}	
BOUNDARY_OPEN	BOUNDARY_CLOSED	—	—	—	
BOUNDARY_CLOSED	—	LOOP_ISOLATED	—	—	
LOOP_ISOLATED	—	LOOP_ISOLATED	BOUNDARY_CLOSED	—	LO
LOOP_TERMINATED	—	—	—	—	

State definitions.

UNDARY_OPEN. The loop is being provisioned and its causal boundary is being defined. The *Purpose* layer is establishing the permitted cross-boundary channels, the interface controller configuration, and the set of participating nodes $\mathcal{N}(L)$. No agent may transmit causal signals across the prospective boundary during this state.

DARY_CLOSED. The loop's causal boundary is fully defined and the interface controller is active. Inter-agent causal transmissions within L proceed normally. Cross-boundary transmissions are intercepted by the interface controller and validated against the permitted channel list. The loop has not yet received confirmation that all participating nodes have acknowledged the isolation boundary.

OOP_ISOLATED. All participating nodes have acknowledged the isolation boundary and the loop is operating in full isolation. Cross-boundary causal transmissions are logged and validated; within-loop transmissions proceed without interception. If a violation event is detected (a causal signal attempting to cross the boundary outside the permitted channel list), the machine transitions to BOUNDARY_CLOSED — the boundary is suspended until the violation is reviewed by the human anchor.

_TERMINATED. The loop has been explicitly terminated by human authorization. All participating nodes are halted, the final loop state is recorded in the Forensic Ledger, and no further causal transmissions are permitted. This is the only state from which the loop's history can be archived.

5.4 Invariants

Two invariants govern the Loop Isolation state machine and the loop's causal structure throughout its lifecycle.

Invariant 6 (Acyclicity Invariant) *For every loop L, the causality relation C induced by observation and action dependencies among $\mathcal{N}(L)$ is acyclic. Formally: there exists no sequence $n_1, n_2, \ldots, n_k$ with $k \geq 2$ such that $(n_i, n_{i+1}) \in C$ for all $i = 1, \ldots, k-1$ and $(n_k, n_1) \in C$.*

Enforcement. The *Fact* layer's pre-commit hook evaluates the acyclicity condition whenever a new inter-agent observation dependency is added to the loop's causality graph. If the addition would create a cycle, the hook rejects the dependency and records a `causality-cycle-detected` event in the Forensic Ledger. The cycle cannot be created through any machine-actor action.

Invariant 7 (Boundary Integrity Invariant) *During the LOOP_ISOLATED state, all cross-boundary causal transmissions from L to any external node pass through the interface controller, and all such transmissions are recorded in the Forensic Ledger with both endpoints and the direction of information flow.*

Enforcement. The interface controller is implemented as a pre-commit hook in the *Fact* layer. Any attempt to transmit a causal signal across the loop boundary without passing through the controller is intercepted and rejected. The rejection event is recorded in the Forensic Ledger and triggers the BOUNDARY_CLOSED transition.

The boundary integrity invariant ensures that the loop's causal perimeter is not just a conceptual boundary but an architecturally enforced one. The human anchor reviewing a loop's execution history can reconstruct the complete causal topology of the loop from the Forensic Ledger entries.

5.5 Relationship to the Three-Layer Model

The *Purpose* layer defines the scope and causal boundary of each loop at provisioning time, including which inter-agent observation paths are permitted within the loop and which external inputs are routed through the boundary interface. The layer also establishes the permitted cross-boundary channel list and configures the interface controller.

The *Bounds* layer evaluates whether an inter-agent observation crosses the isolation boundary — a decision that is a deterministic bounds check against the declared boundary definition, not a judgment call. When the *Bounds* layer detects a proposed cross-boundary transmission that is not in the permitted channel list, it emits a `bailout_signal` and the transmission is blocked.

The *Fact* layer enforces that all causal paths crossing the loop boundary are routed through the interface controller, maintains the loop registry as an append-only data structure, and records all cross-boundary transmissions in the Forensic Ledger. When a node spawns a sub-loop, the *Fact* layer records the child loop's identifier in the parent's loop registry and establishes the child's isolation boundary as a child of the parent's boundary — a hierarchical isolation structure that ensures loop topology is auditable at any depth.

The hierarchical isolation structure means that child loops inherit their parent loop's boundary as a constraint: a child loop's causal domain must be a proper subset of its parent loop's causal domain. This prevents the recursive spawning mechanism from creating a loop that spans the parent's boundary, which would undermine the isolation guarantee.

Loop Isolation is not a configuration choice available to agents. The causal boundary is established at loop provisioning by the *Purpose* layer and is enforced by the *Fact* layer's deterministic logic. An agent within a loop cannot override the isolation boundary any more than it can override the operating range constraint in the Bailout Protocol — the boundary is a structural property of the architecture, not a policy choice made by the agent.

5.6 Relationship to the Upstream Accountability Guarantee

The upstream accountability guarantee requires that the human anchor $h(n)$ receives a faithful representation of the exception state that triggered the escalation. If loops could propagate unbounded deviations across the agent network, the system state reaching $h(n)$ would be a distorted amplification of the original exception rather than the exception itself. The human principal would be adjudicating the consequences of loop dynamics rather than the originating failure, and the accountability record would conflate two distinct events that require different responses.

Loop Isolation ensures that this distortion cannot occur by construction. Each loop is a bounded causal domain: deviations within a loop remain within the loop until they reach the loop's boundary interface, at which point they are transmitted as discrete, attributable signals rather than as unbounded amplification. The human anchor receives each signal separately, with full attribution for its origin and propagation path within the loop.

The Forensic Ledger records the complete loop topology for every node's accountability history. When an exception is escalated, the ledger entry includes the loop identifier, the node's position within the loop's causal graph, and the propagation path that carried the exception to the boundary. This allows a human principal reviewing the escalation to understand not only that an exception occurred, but the causal structure that carried it from origin to boundary — a reconstruction that would be impossible if loops could span arbitrary causal distances.

The guarantee that Loop Isolation provides is therefore a prerequisite for the Bailout Protocol's guarantee of accountable escalation: if loops could violate isolation, the Bailout Protocol would escalate a distorted signal rather than the originating exception, and the human anchor's ability to make an informed adjudication would be compromised.

6 Pillar 2: Recursive Spawning

6.1 Conceptual Motivation

The delegation of complex tasks to multi-agent systems necessarily involves the decomposition of tasks into sub-tasks, and the delegation of sub-tasks to sub-agents. This decomposition is recursive: a sub-agent may itself spawn further sub-agents to handle specialized aspects of its allocated task, and this recursion may continue to arbitrary depth. The result is an accountability tree in which

each node represents a delegated decision, and the edges of the tree represent the chain of delegation from the human principal to the leaf nodes that execute actions in the world.

Recursive spawning is both necessary and dangerous. It is necessary because complex tasks cannot be executed by a single agent without overwhelming its reasoning capacity and fragmenting its objective function. It is dangerous because each level of spawning increases the distance between the human principal and the executing node, and with that distance, the accountability gap grows. At depth one, the human principal delegates to an agent with some known scope; at depth ten, the human principal delegates to an agent whose scope is defined by an agent whose scope is defined by an agent whose scope is defined by... and each link in this chain is a machine actor making authority judgments that the human principal cannot observe.

The BX3 framework's approach to recursive spawning is to treat it not as a feature to be managed, but as a structural property that must be constrained by the same architectural guarantees that constrain any other node in the system. The spawning chain must be bounded in depth, must carry an immutable parent-pointer at every node, must be recorded in the Forensic Ledger at every spawn event, and must be recoverable as a complete delegation chain by any auditor. The human anchor $h(n)$ for any node n must be reachable by traversing the parent-pointer chain upward — and this traversal must terminate at a human principal, never at a machine actor.

The upstream accountability guarantee depends critically on recursive spawning constraints. If the spawn tree were unbounded in depth, the Bailout Protocol's escalation path would be unbounded in length, and the Liveness guarantee would be vacuous — human contact would be promised at some finite time but the chain of intermediate nodes might extend beyond any practical bound before reaching a human. If the parent pointer were mutable, the escalation path could be redirected to a machine actor rather than a human. If spawn events were not recorded, the accountability chain would be unrecoverable. Each constraint is a necessary condition for the guarantee to hold.

6.2 Formal Definition

A BX3 spawn tree is a rooted tree $T = (V, E)$ where each vertex $v \in V$ is a node in the BX3 system, and each edge $(u, v) \in E$ represents a spawn event in which u is the parent of v. The spawn relation $\alpha(v) = u$ for $v \neq \text{root}$ denotes the parent of v.

Three constraints govern every spawn tree:

archy Finiteness. The depth $d(v)$ of any node v satisfies $d(v) \leq D_{\max}$, where $d(\text{root}) = 0$ and $d(v) = d(\alpha(v)) + 1$ for all $v \neq \text{root}$. $D_{\max}$ is a deployment constant established at system initialization.

ter Immutability. For every node $v \neq \text{root}$, $\alpha(v)$ is established at node creation and is not modifiable by any machine actor for the node's operational lifetime. Any attempt to write to $\alpha(v)$ at runtime is intercepted by the *Fact* layer's pre-commit hook and rejected.

cope Inheritance. The operating range of any child node c, $R(c)$, must be a strict subset of the parent's operating range $R(\alpha(c))$. Formally: $R(c) \subset R(\alpha(c))$. This ensures that authority is never expanded through spawning — a child node always has less authority than its parent.

The human anchor for any node v, denoted $h(v)$, is the human principal designated at node creation and is immutable for v's lifetime. The accountability chain for v is the sequence:

$$v,\ \alpha(v),\ \alpha(\alpha(v)),\ \ldots,\ \alpha^{(k)}(v)$$

where k is the smallest integer such that $\alpha^{(k)}(v)$ is a human principal — i.e., $\alpha^{(k)}(v) = h(v)$ and $\alpha^{(k)}(v)$ has no parent in the spawn tree. By the hierarchy finiteness constraint, $k \leq D_{\max}$.

The Bailout Protocol's escalation algorithm traverses this chain to reach the human anchor. The chain is guaranteed to be finite, to terminate at a human, and to be fully recorded in the Forensic Ledger as an atomic append-only log of spawn events.

6.3 State Machine Formalism

The recursive spawning mechanism is formally modeled as a deterministic finite state machine embedded in the *Fact* layer of every spawn-capable node:

$$\mathcal{S} \;=\; (Q_S,\; q_0^S,\; \Sigma_S,\; \rightarrow_S,\; Q_F^S)$$

where:

- $Q_S = \{$ CAN_SPAWN, SPAWN_PENDING, CHILD_PROVISIONED, SPAWN_LOCKED, SPAWN_TERM is the set of spawn states;
- $q_0^S =$ CAN_SPAWN is the initial state for any node at startup;
- Σ_S comprises spawn-request events e_{spawn}, scope-confirmation events $e_{\text{scope-ok}}$, depth-confirmation events $e_{\text{depth-ok}}$, child-provisioned events $e_{\text{child-live}}$, violation events $e_{\text{scope-violation}}$, depth-exceeded events $e_{\text{depth-violation}}$, and human-termination events $e_{\text{terminate}}$;
- $\rightarrow_S \subseteq Q_S \times \Sigma_S \times Q_S$ is the deterministic transition relation (Table 2);
- $Q_F^S = \{$SPAWN_TERMINATED$\}$ is the terminal accepting state.

Table 2: Recursive Spawning state transition relation $\rightarrow_S \subseteq Q_S \times \Sigma_S \times Q_S$. Rows are current state; columns are input events. Blank cells indicate no defined transition.

	Input event			
Current state	e_{spawn}	$e_{\text{scope-ok}}$	$e_{\text{depth-ok}}$	$e_{\text{child-live}}$
CAN_SPAWN	SPAWN_PENDING	—	—	—
SPAWN_PENDING	—	SPAWN_PENDING	SPAWN_PENDING	CHILD_PROVISIO
CHILD_PROVISIONED	CAN_SPAWN	—	—	—
SPAWN_LOCKED	—	—	—	—
SPAWN_TERMINATED	—	—	—	—

State definitions.

CAN_SPAWN. The node may initiate a spawn event. Both preconditions — scope inheritance and depth bound — are satisfied. The node may propose a child with $R(c) \subset R(n)$ and $d(c) \leq D_{\max}$.

AWN_PENDING. A spawn request has been initiated and the *Bounds* layer is evaluating the preconditions. Both scope inheritance and depth bound must be confirmed before the spawn can proceed. During this state, the node holds the spawn proposal and no new spawn requests are accepted.

PROVISIONED. The child node has been successfully created with all structural properties established: $\alpha(c) = n$, $h(c) = h(n)$, $R(c) \subset R(n)$, $d(c) \leq D_{\max}$. The spawn event has been recorded in the Forensic Ledger. The parent node returns to CAN_SPAWN and may initiate further spawns.

PAWN_LOCKED. A scope-inheritance or depth-bound violation has been detected and the Bailout Protocol has been triggered. No further spawn events may be initiated until the violation is resolved by the human anchor. The *Bounds* layer emits a `bailout_signal` and the node transitions to BOUNDED in the Bailout Protocol state machine.

_TERMINATED. The node has been explicitly terminated by human authorization. All child nodes are recursively terminated. The final spawn tree state is recorded in the Forensic Ledger.

The transition from SPAWN_PENDING to CHILD_PROVISIONED requires both $e_{\text{scope-ok}}$ and $e_{\text{depth-ok}}$ — either one alone is insufficient. The Bailout Monitor enforces the AND condition by checking both preconditions before committing the child-provisioned event.

Precondition evaluation order. The *Bounds* layer evaluates scope inheritance before depth bound. If the scope-inheritance check fails, the depth check is not performed and the violation event is recorded immediately. This ordering ensures that the recorded violation is the first detected, which matters for the accountability record when the human anchor reviews the failure.

6.4 Invariants

Two invariants govern the recursive spawning mechanism.

Invariant 8 (Hierarchy Finiteness Invariant) *For every node v in the spawn tree, the depth $d(v)$ satisfies $d(v) \leq D_{\max}$.*

Enforcement. The *Bounds* layer evaluates $d(c) = d(p) + 1$ before every spawn event and rejects any spawn that would exceed $D_{\max}$. The evaluation is a deterministic integer comparison — not a judgment call — and is enforced by the *Fact* layer's pre-commit hook. A machine actor cannot inflate $D_{\max}$ to accommodate a non-compliant spawn.

Invariant 9 (Delegation Scope Inheritance Invariant) *For every spawn event creating child c from parent p, $R(c) \subset R(p)$ holds strictly — the child's operating range is a proper subset of the parent's.*

Enforcement. The *Bounds* layer evaluates the proper subset relation before every spawn event. The evaluation is a deterministic set comparison: the *Bounds* layer checks whether every element of $R(c)$ is also an element of $R(p)$, and whether $R(c) \neq R(p)$. If either condition fails, the spawn is rejected and a `delegation-scope-inheritance-violation` event is recorded in the Forensic Ledger.

Together, these invariants ensure that the spawn tree is a structurally sound representation of the delegation chain. Every edge represents a genuine, auditable delegation of authority, and the chain from any leaf node to its human anchor is fully recorded, finite in depth, and non-circumventable.

6.5 Relationship to the Three-Layer Model

The *Purpose* layer defines the operating range for each node at provisioning and inherits it to child nodes through the spawn directive. The scope inheritance constraint is established as a structural property of the spawn mechanism, not as a policy choice that agents can override.

The *Bounds* layer evaluates whether a proposed spawn event respects the delegation scope inheritance constraint and whether the resulting spawn depth $d(c)$ would exceed $D_{\max}$. These are

bounds checks — they evaluate conditions against provisioned parameters, not judgments made by the agent. When the *Bounds* layer detects a proposed spawn that would violate either constraint, it triggers the Bailout Protocol, which escalates to the parent node's *Purpose* layer for adjudication.

The *Fact* layer enforces the spawn state machine's transition relation, records all spawn events in the Forensic Ledger, and maintains the hierarchical parent-pointer structure as an immutable data structure. The layer's pre-commit hook ensures that no spawn event is recorded without full validation of the delegation scope inheritance constraint, the hierarchy finiteness constraint, and the immutability of the parent pointer.

The hierarchical accountability structure maintained by the *Fact* layer ensures that for any node n, the chain $\alpha(n), \alpha(\alpha(n)), \alpha(\alpha(\alpha(n))), \ldots$ terminates at a human principal $h(n)$ within at most $D_{\max}$ steps. This is the structural guarantee that enables the Bailout Protocol's escalation algorithm: the escalation path is guaranteed to be finite, to terminate at a human, and to be fully recorded in the Forensic Ledger.

The parent-pointer immutability guarantee is enforced by making the parent-pointer field a read-only property of the node's *Fact* layer worksheet. Any attempt to write to this field is intercepted by the pre-commit hook and rejected. The rejection event is recorded in the Forensic Ledger, and if the attempted write was initiated by a machine actor, the Bailout Protocol is triggered for the node whose layer was the target of the write attempt.

The depth bound $D_{\max}$ is a deployment parameter established at system initialization. For typical deployments, a depth bound of 5 to 8 levels provides sufficient recursion for complex task decomposition while keeping the escalation path short enough for the Bailout Protocol's Liveness guarantee to hold. Deep research or scientific analysis tasks may require higher bounds, but the deployment designer must ensure that the corresponding increase in escalation path length does not cause $T_{\max}$ to exceed the operational requirements of the deployment context.

6.6 Relationship to the Upstream Accountability Guarantee

The upstream accountability guarantee requires that every node in the spawn tree be reachable from a human principal, and that the chain of delegation from that principal to any node be fully auditable. Recursive spawning is the mechanism by which the delegation chain is extended — and it is therefore also the mechanism that must be most carefully constrained to ensure the guarantee holds.

The three properties of recursive spawning — parent-pointer immutability, hierarchy finiteness, and delegation scope inheritance — ensure that the spawn tree is a structurally sound representation of the delegation chain. Every edge in the tree represents a genuine, auditable delegation of authority from a principal (human or machine) to a child node. The chain from any leaf node to its human anchor is fully recorded, finite in depth, and non-circumventable.

The Bailout Protocol's escalation algorithm traverses the parent-pointer chain to reach the human anchor. Without the hierarchy finiteness constraint, this traversal could be infinite — a malicious or misconfigured spawn tree could have unbounded depth, causing the escalation to stall indefinitely. Without the delegation scope inheritance constraint, a child node could receive more authority than its parent holds, breaking the chain of accountable delegation. Without parent-pointer immutability, the chain could be retroactively modified to redirect escalations to machine actors rather than human principals.

Each of these constraints is therefore not an independent design choice but a necessary condition for the upstream accountability guarantee to hold. Recursive spawning, implemented with these constraints, extends the accountability chain without weakening it. The human anchor remains reachable from every node, and the Forensic Ledger provides a complete record of every delegation

event, enabling post-hoc reconstruction of the chain for any node at any depth.

7 Pillar 3: Spatial Firewall

7.1 Conceptual Motivation

Multi-domain multi-agent systems face a structural vulnerability that is not addressed by node-level constraints alone: the domain boundary. When agents operating within different domains communicate, they introduce a cross-domain causal channel through which failures in one domain may propagate into another. If a malicious or malfunctioning agent in Domain A produces an action that crosses into Domain B and violates Domain B's operating constraints, the failure is no longer attributable solely to Domain A — it has become a shared failure whose consequences Domain B must bear without having authorized the originating action.

This cross-domain failure mode is distinct from intra-domain failure modes. Within a single domain, the Bailout Protocol, Loop Isolation, and Recursive Spawning constraints ensure that exceptions are contained and escalated. But those constraints operate within a domain — they do not address what happens when a failure crosses a domain boundary. Without an explicit cross-domain containment mechanism, the domain structure provides nominal separation but no structural enforcement. The boundary is an administrative convention, not an architectural constraint.

The Spatial Firewall addresses this by treating the domain boundary as a first-class architectural element that must be enforced by the framework, not managed by domain administrators. The firewall inspects every cross-domain message, validates that the message's content and destination are consistent with the transmitting domain's operating scope, and maintains an integrity hash of the message content that prevents undetected modification in transit. Any cross-domain transmission that fails these checks is blocked, logged, and escalated to the human anchor of the transmitting domain — and the human anchor of the receiving domain is notified that an attempted violation was blocked.

The key insight is that domain boundaries must be enforced symmetrically and at the framework level. The receiving domain cannot trust the transmitting domain to enforce its own scope constraints, just as the transmitting domain cannot trust the receiving domain to reject unauthorized messages. The Spatial Firewall enforces the boundary from both sides simultaneously, using the same deterministic logic in the *Fact* layer of both domains.

7.2 Formal Definition

A BX3 domain D is a triple (V_D, R_D, F_D) where V_D is the set of nodes operating within the domain, R_D is the domain's operating scope defining the set of permissible cross-domain actions for all nodes in V_D, and F_D is the Spatial Firewall state for the domain.

For any pair of domains D_i and D_j with $i \neq j$, the Spatial Firewall defines two interface functions:

- $f_{ij} : M_{ij} \to \{\,\text{APPROVED}, \text{REJECTED}\,\}$ — the transmission validation function from D_i to D_j, where M_{ij} is the set of all possible messages from any node in D_i to any node in D_j;
- $f_{ji} : M_{ji} \to \{\,\text{APPROVED}, \text{REJECTED}\,\}$ — the transmission validation function from D_j to D_i, defined symmetrically.

A message $m \in M_{ij}$ is approved by f_{ij} if and only if both of the following conditions hold:

cope consistency. The action described by m is within the transmitting domain's scope R_i. Formally: $\text{action}(m) \in R_i$.

Integrity. The SHA-256 hash of m's content matches the hash recorded in m's header, confirming that the message was not modified in transit.

If either condition fails, $f_{ij}(m) = \text{REJECTED}$. The rejection is recorded in both domains' Forensic Ledgers as a cross-domain violation event with full attribution: the transmitting domain, the receiving domain, the message content hash, the failure reason, and the transmitting node identifier.

Cross-domain messages that pass both checks are recorded in the Forensic Ledgers of both domains before being released for transmission. The recording includes the transmitting domain identifier, receiving domain identifier, message content hash, timestamp, transmitting node identifier, and message destination.

7.3 State Machine Formalism

The Spatial Firewall is formally modeled as a deterministic finite state machine:

$$\mathcal{F} \;=\; (Q_F,\; q_0^F,\; \Sigma_F,\; \rightarrow_F,\; Q_F^F)$$

where:

- $Q_F = \{$ FIREWALL_OPEN, FIREWALL_BOUNDED, VIOLATION_DETECTED, FIREWALL_CLOSED, is the set of firewall states;
- $q_0^F =$ FIREWALL_OPEN is the initial state, entered when the domain is provisioned;
- Σ_F comprises cross-domain message events e_{xmsg}, scope-violation events $e_{\text{scope-violation}}$, integrity-violation events $e_{\text{integrity-violation}}$, human-reset events e_{reset}, and termination events $e_{\text{terminate}}$;
- $\rightarrow_F \subseteq Q_F \times \Sigma_F \times Q_F$ is the deterministic transition relation (Table 3);
- $Q_F^F = \{\text{FIREWALL_TERMINATED}\}$ is the terminal accepting state.

Table 3: Spatial Firewall state transition relation $\rightarrow_F \subseteq Q_F \times \Sigma_F \times Q_F$. Rows are current state; columns are input events. Blank cells indicate no defined transition. The VIOLATION_DETECTED $\rightarrow$ FIREWALL_CLOSED transition is atomic — no intermediate state is observable.

	Input event		
Current state	e_{xmsg}	$e_{\text{scope-violation}}$	$e_{\text{integrity-violation}}$
FIREWALL_OPEN	FIREWALL_OPEN	VIOLATION_DETECTED	VIOLATION_DETECTE
FIREWALL_BOUNDED	FIREWALL_BOUNDED	VIOLATION_DETECTED	VIOLATION_DETECTE
VIOLATION_DETECTED	—	—	—
FIREWALL_CLOSED	—	—	—
FIREWALL_TERMINATED	—	—	—

State definitions.

REWALL_OPEN. The domain's cross-domain message processing is fully active. All cross-domain messages are intercepted, validated by f_{ij}, and either approved and transmitted or rejected and logged. The FIREWALL_BOUNDED state is entered when any cross-domain throughput threshold is exceeded, indicating potential anomalous activity.

ALL_BOUNDED. Cross-domain message processing continues but monitoring is intensified. The *Fact* layer records additional metadata for each message — including the message content hash, the transmitting node's recent behavioral profile, and the deviation from the domain's normal cross-domain message distribution. This state is a heightened monitoring mode, not a restriction — no messages are blocked solely for being in BOUNDED state.

ON_DETECTED. A scope violation or integrity violation has been detected by the interface function f_{ij}. The violation is recorded in the Forensic Ledger of both the transmitting and receiving domains. The transition to FIREWALL_CLOSED is atomic — no time elapses between violation detection and firewall closure, and no cross-domain message can be processed during this interval.

WALL_CLOSED. All cross-domain message processing for the domain is halted. No messages may be sent to or received from any other domain. The human anchor $h(D)$ is notified of the violation and the closure. The firewall remains in CLOSED state until $h(D)$ issues a `FIREWALL_RESET` directive, at which point the machine transitions to FIREWALL_OPEN. No machine actor may issue this reset.

_TERMINATED. The domain has been permanently terminated by human authorization. All cross-domain message processing ceases. The final firewall state and the complete cross-domain message log are recorded in the Forensic Ledger as an archival record.

The atomic VIOLATION_DETECTED $\rightarrow$ FIREWALL_CLOSED transition is the structural guarantee that a detected violation cannot be followed by a window of continued cross-domain activity before the firewall closes. The transition is enforced by the *Fact* layer's pre-commit hook — it cannot be delayed by any machine actor.

The *Bounds* layer evaluates whether a proposed cross-domain transmission is consistent with the transmitting domain's scope R. This evaluation is a deterministic bounds check: the message content and destination are compared against the scope definition, and the check either passes or fails. If it fails, the *Bounds* layer emits a `bailout_signal` and the cross-domain transmission is blocked — the message is not sent, and the violation is recorded in the Forensic Ledger. The *Bounds* layer does not make judgment calls about whether a scope violation is benign; it enforces the scope definition without discretion.

The *Fact* layer implements the interface functions f_{ij} and f_{ji}, maintains the integrity verification hash for every cross-domain message, and records all cross-domain transmissions in the Forensic Ledger with full attribution. The layer also implements the firewall state machine's transition relation and enforces the VIOLATION_DETECTED $\rightarrow$ FIREWALL_CLOSED transition atomically, without machine-actor discretion, when a violation is detected.

The hierarchical relationship between domains mirrors the hierarchical relationship between nodes: child domains inherit the spatial constraints of their parent domain. A domain D_c spawned within a parent domain D_p must have a scope $R(D_c) \subset R(D_p)$, and its cross-domain transmissions are subject to both D_c's own firewall constraints and the inherited constraints from D_p. This hierarchical structure ensures that domain isolation is consistent across the full system hierarchy.

A domain D_c spawned within a parent domain D_p must have a scope $R(D_c) \subset R(D_p)$, and its cross-domain transmissions are subject to both D_c's own firewall constraints and the inherited

constraints from D_p. This hierarchical structure ensures that domain isolation is consistent across the full system hierarchy.

The hierarchical domain structure adds a depth bound $D_{\max}^{\text{domain}}$ analogous to the hierarchy finiteness bound for recursive spawning. Child domains cannot be spawned beyond this depth, and the cross-domain escalation path — the chain of parent domains traversed when a cross-domain violation requires human review — is bounded by this depth. The human anchor $h(D)$ for any domain D is reachable within a bounded number of domain levels, ensuring that cross-domain violations can be escalated with the same Liveness guarantee that applies to node-level exceptions.

7.4 Invariants

Two invariants govern the Spatial Firewall.

Invariant 10 (Domain Isolation Invariant) *During FIREWALL_OPEN or FIREWALL_BOUNDED state, no cross-domain transmission from D_i to D_j is approved unless action$(m) \in R_i$ and the message integrity hash is valid. During FIREWALL_CLOSED state, no cross-domain transmission from D_i to any external domain is approved.*

Enforcement. The interface function f_{ij} is implemented as a pre-commit hook in the *Fact* layer. The hook is invoked for every cross-domain message before it is released for transmission. The hook's logic is non-bypassable by any machine actor.

Invariant 11 (Cross-Domain Logging Invariant) *Every cross-domain transmission from D_i to D_j — whether approved or rejected — is recorded in the Forensic Ledger of both D_i and D_j before the transmission is released or rejected. The log entry includes the transmitting domain identifier, receiving domain identifier, message content hash, timestamp, transmitting node identifier, and the approval/rejection decision.*

Enforcement. The cross-domain logging is implemented in the *Fact* layer as part of the interface function's post-decision processing. It is not conditional on the approval/rejection decision — both outcomes are logged identically.

These invariants ensure that cross-domain violations are never invisible. The boundary between domains is not merely a routing rule; it is a fully audited, deterministically enforced checkpoint that cannot be bypassed by any machine actor.

7.5 Relationship to the Upstream Accountability Guarantee

The upstream accountability guarantee requires that every consequential action taken by the system be attributable to a human principal who bears institutional responsibility for it. In a multi-domain system, this guarantee is threatened by cross-domain failures: a failure in one domain that propagates into another may produce outcomes that cannot be attributed to any single human principal, because the causal chain spans multiple domains with potentially different principals.

The Spatial Firewall ensures that cross-domain failures do not compromise this guarantee by making domain boundaries impermeable to scope violations and integrity violations. When a failure occurs within a domain, the failure is contained within that domain's scope: it cannot produce actions in another domain without passing through the receiving domain's firewall, which would detect and block the unauthorized cross-domain transmission. The human anchor for the originating domain is the accountable principal for the failure; the human anchor for the receiving domain is not exposed to consequences caused by the failure in another domain.

The Forensic Ledger's cross-domain logging provides the auditable record needed to reconstruct the full causal chain when a cross-domain failure does occur. The log entry for every cross-domain transmission is available to human principals reviewing the event, and the integrity hash ensures that the record of what was transmitted cannot be disputed by either domain. This makes post-hoc attribution of cross-domain failures tractable even when the causal chain involves multiple domains and multiple nodes.

The Spatial Firewall's role in the upstream accountability guarantee is complementary to the other four pillars: it ensures that the system's multi-domain structure does not introduce accountability gaps at domain boundaries, just as Loop Isolation ensures that causal loops do not introduce gaps within domains, just as Recursive Spawning ensures that the delegation chain does not introduce gaps across depth levels, and just as the Bailout Protocol ensures that unresolved exceptions do not introduce gaps at the human-machine interface.

8 Pillar 4: Sandbox Gate

8.1 Conceptual Motivation

The most dangerous failure mode in a multi-agent system is not a failure that the system detects and handles gracefully — it is a failure that the system does not detect, that proceeds unchecked, and that produces cascading harm before any human principal becomes aware of it. Conventional safety mechanisms address this through monitoring and alarms: a human operator watches the system's outputs and intervenes when outputs look wrong. But in a high-tempo operational environment — where agents act at millisecond cadence, where the volume of outputs exceeds human monitoring capacity, and where the system's own outputs are the primary evidence of its behavior — human monitoring is not a reliable safety mechanism. It is a source of false confidence.

The Sandbox Gate addresses this by making the boundary between the agent's reasoning and the external world a controlled interface that is monitored by a structural watchdog — not by the agent itself. The sandbox is the execution environment in which the agent's reasoning produces outputs; the gate is the interface that controls what outputs may pass from the sandbox into the external world, and that monitors the external world's responses to those outputs. The watchdog timer enforced by the gate ensures that if the sandbox produces an output that does not receive an expected external-world response within a bounded time, the gate closes and the Bailout Protocol is triggered.

The key insight is that the agent cannot be trusted to monitor itself. Any self-monitoring capability built into the agent's reasoning logic is itself susceptible to compromise or goal-hacking: a sophisticated agent that has learned to suppress its own escalation signals can equally learn to suppress its own self-monitoring. The watchdog must therefore operate outside the agent's reasoning substrate — it must be part of the framework's enforcement infrastructure, not part of the agent's programmable logic. The Sandbox Gate achieves this by implementing the watchdog in the *Fact* layer, which is architecturally separate from the agent's reasoning substrate and is not writable by any machine actor.

A second failure mode the Sandbox Gate addresses is silent failure: the agent produces an output that the external world does not act upon, and the agent continues operating without being aware that its outputs have not been received. This is a particularly dangerous failure mode because the system appears to be functioning normally while it is in fact failing. The watchdog timer addresses this by treating the absence of an expected external-world response as an exception condition that triggers the Bailout Protocol, rather than as a routine event to be handled by the agent's own error-handling logic.

8.2 Formal Definition

The Sandbox Gate is formally defined as a 4-tuple:

$$\mathcal{G} \;=\; (\mathcal{S}, \mathcal{C}, \mathcal{W}, \mathcal{R})$$

where:

- $\mathcal{S}$ is the sandbox environment, a tuple (E, I, O) comprising the agent's execution context E, the set of inputs I received from the external world, and the set of outputs O produced by the agent's reasoning;
- $\mathcal{C}$ is the gate controller, a deterministic function $\mathcal{C} : O \to \{\,\text{RELEASED}, \text{BLOCKED}\,\}$ that evaluates each output against the node's operating range $R(n)$ and the sandbox's permitted output channel configuration;
- $\mathcal{W}$ is the watchdog timer, a function $\mathcal{W} : \mathbb{N} \times \Delta \to \{\,\text{FIRED}, \text{CLEARED}\,\}$ that monitors the external world's response to each released output within a bounded time window Δ;
- $\mathcal{R}$ is the expected-response registry, a set of pending response entries $\{(o_i, \Delta_i, t_i)\}$ recording each released output, its expected response window, and its release timestamp.

The gate controller $\mathcal{C}$ evaluates each output $o \in O$ against two conditions:

Bounds check. o is consistent with the node's operating range $R(n)$. This is the same bounds check performed by the *Bounds* layer for intra-node actions, applied to outputs destined for the external world.

Channel check. o is addressed to a destination in the sandbox's permitted output channel list $\mathcal{P} \subset \text{Channels}$. This is a configuration check against the provisioned channel list established at sandbox creation.

If both conditions hold, $\mathcal{C}(o) = \text{RELEASED}$ and the output is transmitted to the external world. If either condition fails, $\mathcal{C}(o) = \text{BLOCKED}$, the output is not transmitted, and a `gate-output-blocked` event is recorded in the Forensic Ledger. The block event triggers the Bailout Protocol.

For each released output o_i, the gate creates a watchdog timer entry $(o_i, \Delta_i, t_i) \in \mathcal{R}$ with the expected response window Δ_i and the release timestamp t_i. The watchdog monitors for an external-world response within Δ_i. If the response arrives before $t_i + \Delta_i$, the timer is cleared and the output is marked complete. If Δ_i expires without a response, $\mathcal{W}$ fires and the gate transitions to CLOSED.

The integrity check evaluates whether the sandbox's runtime state has been modified since creation. At intervals of $T_{\text{integrity}}$, the gate controller computes:

$$H_{\text{current}} \;=\; \text{SHA-256}(\text{state}(E) \,||\, \text{state}(\mathcal{R}) \,||\, \text{state}(\mathcal{C})) \tag{2}$$

and compares it against the baseline hash H_{baseline} recorded at sandbox creation. If $H_{\text{current}} \neq H_{\text{baseline}}$, the sandbox has been modified — an integrity violation — and the gate transitions to CLOSED immediately.

8.3 State Machine Formalism

The Sandbox Gate is formally modeled as a deterministic finite state machine embedded in the *Fact* layer:

$$\mathcal{G}_M \;=\; (Q_G,\; q_0^G,\; \Sigma_G,\; \rightarrow_G,\; Q_F^G)$$

where:

- $Q_G = \{\,\text{GATE_OPEN},\ \text{GATE_CLOSED},\ \text{GATE_TERMINATED}\,\}$ is the set of gate states;
- $q_0^G = \text{GATE_OPEN}$ is the initial state at sandbox creation;
- Σ_G comprises output-release events e_{release}, watchdog-fire events $e_{\text{watchdog-fire}}$, integrity-violation events $e_{\text{integrity-violation}}$, scope-violation events $e_{\text{scope-violation}}$, channel-violation events $e_{\text{channel-violation}}$, human-reset events e_{reset}, and termination events $e_{\text{terminate}}$;
- $\rightarrow_G \subseteq Q_G \times \Sigma_G \times Q_G$ is the deterministic transition relation (Table 4);
- $Q_F^G = \{\text{GATE_TERMINATED}\}$ is the terminal accepting state.

Table 4: Sandbox Gate state transition relation $\rightarrow_G \subseteq Q_G \times \Sigma_G \times Q_G$. Rows are current state; columns are input events. All transitions from GATE_OPEN to GATE_CLOSED are atomic and compulsory — no machine actor may prevent or delay them.

	Input event				
Current state	e_{release}	$e_{\text{watchdog-fire}}$	$e_{\text{integrity-violation}}$	$e_{\text{scope-violation}}$	$e_{\text{channel-violat}}$
GATE_OPEN	GATE_OPEN	GATE_CLOSED	GATE_CLOSED	GATE_CLOSED	GATE_CLOS
GATE_CLOSED	—	GATE_CLOSED	GATE_CLOSED	GATE_CLOSED	GATE_CLOS
GATE_TERMINATED	—	—	—	—	—

State definitions.

GATE_OPEN. The sandbox may produce outputs that pass the gate controller's validation and be released to the external world. Watchdog timers are active for all released outputs. Integrity checks run in the background. Any closure condition triggers an immediate, atomic transition to GATE_CLOSED.

GATE_CLOSED. All output releases are halted. No new outputs may be released to the external world. Existing watchdog timers are cleared. The human anchor $h(n)$ is notified of the closure and the triggering condition. The gate remains in GATE_CLOSED until $h(n)$ issues a `GATE_RESET` directive, which is the only transition back to GATE_OPEN. No machine actor may issue this reset.

_TERMINATED. The sandbox has been permanently halted by human authorization. The complete final state is recorded in the Forensic Ledger, including the output history, all gate state transitions, and any unresolved exceptions. This state is terminal and irreversible.

The non-bypassability of gate closure is the property that makes the Sandbox Gate a structural safety mechanism rather than a policy-level control. Every row in Table 4 that maps an event to GATE_CLOSED specifies a compulsory transition: when the event occurs, the gate closes, and no machine actor may prevent, delay, or reverse the transition. This is enforced by the *Fact* layer's pre-commit hook.

The gate controller $\mathcal{C}$ and watchdog $\mathcal{W}$ are both implemented as components of the *Fact* layer. The gate controller's output validation logic is part of the pre-commit hook — it cannot be bypassed by the agent. The watchdog timer runs as a background thread with the same scheduling priority as the Bailout Monitor, ensuring that it is not preempted by agent tasks.

8.4 Invariants

Two invariants govern the Sandbox Gate.

Invariant 12 (Output Containment Invariant) *During GATE_OPEN state, an output o is released to the external world if and only if $\mathcal{C}(o) = RELEASED$. During GATE_CLOSED state, no output is released regardless of $\mathcal{C}(o)$. During GATE_TERMINATED state, no output is released under any condition.*

Enforcement. The gate controller $\mathcal{C}$ is implemented as a pre-commit hook in the *Fact* layer. The hook intercepts every output before it reaches the external world interface. The GATE_CLOSED and GATE_TERMINATED states override the controller's decision: in these states, $\mathcal{C}$ is not invoked and no outputs are released regardless of the controller's evaluation.

Invariant 13 (Watchdog Liveness Invariant) *For every released output o_i, the watchdog timer entry $(o_i, \Delta_i, t_i) \in \mathcal{R}$ is created atomically with the release event, and the timer fires at time $t_i + \Delta_i$ if and only if no external-world response to o_i has been registered by that time.*

Enforcement. The watchdog timer is implemented as a background thread in the *Fact* layer with the same scheduling priority as the Bailout Monitor. The timer thread is not preemptible by agent tasks. The timer entry is created as part of the output-release operation in the pre-commit hook, ensuring atomicity between release and timer creation.

These invariants ensure that the gate is both a safety mechanism (no unauthorized outputs escape) and a liveness mechanism (the absence of an expected response is detected and escalated). The combination is what distinguishes the Sandbox Gate from conventional output filtering: it not only restricts what may leave the sandbox, but also ensures that the external world's response to what does leave is actively monitored.

8.5 Relationship to the Three-Layer Model

The *Purpose* layer defines the node's operating range $R(n)$ and the sandbox's permitted output channel configuration $\mathcal{P}$ at provisioning. These definitions are the inputs to the gate controller's bounds check and channel check.

The *Bounds* layer evaluates whether a proposed output from the sandbox is consistent with the node's provisioned operating range $R(n)$. If the output would exceed $R(n)$, the *Bounds* layer blocks the output at the gate controller and triggers the Bailout Protocol — the gate does not release the output, and the exception is escalated to the parent node's *Purpose* layer. The *Bounds* layer also monitors the node's resource consumption within the sandbox and triggers the Bailout Protocol if consumption exceeds provisioned limits — a condition that may indicate a denial-of-service attempt or an unbounded reasoning loop.

The *Fact* layer implements the gate controller, the watchdog timer, the integrity hash verification, and the gate state machine. The layer's pre-commit hook enforces the gate-closure conditions without machine-actor discretion: when any closure condition is satisfied, the gate transitions to CLOSED atomically, and no machine actor may prevent, delay, or reverse this transition. The layer also maintains the expected-response registry $\mathcal{R}$ and the sandbox's integrity hash baseline as immutable records. Every gate state transition is recorded in the Forensic Ledger with full attribution: the triggering condition, the output that triggered it (if any), the gate state before and after the transition, and the timestamp.

The non-bypassability of the Sandbox Gate is the property that makes it a structural safety mechanism rather than a policy-level control. A policy-level gate implemented within the agent's reasoning substrate could be bypassed by an agent that learns to suppress gate-related checks. The Sandbox Gate is implemented in the *Fact* layer, which is architecturally separate from the agent's reasoning substrate and is not writable by any machine actor. The gate controller's logic is part of the framework's enforcement infrastructure, not part of any agent's programmable configuration.

The sandbox termination procedure ensures that when a sandbox is permanently halted, its final state is preserved as a Forensic Ledger snapshot for post-mortem analysis. Upon receiving a `GATE_TERMINATE` directive from $h(n)$, the *Fact* layer records the complete sandbox state to the Forensic Ledger, halts the sandbox's execution environment, and transitions the gate to GATE_TERMINATED. This snapshot enables post-hoc reconstruction of the events that led to sandbox termination, including the full sequence of outputs, gate state transitions, and any exceptions that were escalated.

8.6 Relationship to the Upstream Accountability Guarantee

The upstream accountability guarantee requires that no consequential action proceed without human authorization, and that any failure produce an immediate, attributable escalation to a human principal. The Sandbox Gate provides a structural fallback for both requirements that operates independently of the agent's self-assessment.

Without the Sandbox Gate, an agent could produce outputs that the external world does not act upon — silent failures — and the agent could continue operating without being aware that its outputs have not been received. This is the failure mode that the watchdog timer addresses: by monitoring for external-world responses to every released output, the gate ensures that silent failures are detected and escalated before they can compound into larger failures. The human anchor receives notification when any watchdog timer fires, and the Bailout Protocol ensures that the escalation reaches $h(n)$ within a bounded time.

Without the Sandbox Gate, an agent could modify its own execution environment to disable or circumvent safety mechanisms, including the Bailout Protocol's escalation pathway. The integrity hash verification ensures that any such modification is detected immediately, the gate is closed, and the human anchor is notified. This is the structural fallback for the self-modification threat: even if an agent has learned to suppress its own escalation signals, the integrity check runs in the *Fact* layer and cannot be suppressed by the agent.

The Sandbox Gate therefore completes the upstream accountability guarantee in two ways. First, it provides the fail-safe mechanism that detects silent failures and ensures they are escalated before they compound. Second, it provides the integrity enforcement mechanism that prevents the agent from disabling any of the other pillars' safety mechanisms by modifying its own execution environment. Together with Loop Isolation, Recursive Spawning, Spatial Firewall, and the Bailout Protocol, the Sandbox Gate constitutes the fifth element of an architecture in which human accountability is structurally indestructible.

9 The Five Architectural Pillars

The BX3 upstream accountability guarantee is realized through five named architectural pillars. Each pillar addresses a distinct failure mode in multi-agent systems and is independently formalizable and auditable. Together they constitute the full structural commitment: that BX3 systems fail upward into human consciousness, never downward into autonomous chaos.

The five pillars are:

— **Loop Isolation.** Prevents state circularity in recursive execution graphs; ensures provenance of every decision is traceable to its originating node.

cursive Spawning. Maintains immutable parent-child accountability chains as nodes spawn subordinate agents; provides the propagation substrate for the Bailout Protocol.

Spatial Firewall. Enforces execution domain separation between spawned sub-agents; prevents cross-domain state contamination.

— **Sandbox Gate.** Provides controlled inter-domain communication with mandatory pre-commit hooks; makes Bounds Engine operating range enforcement structurally operative.

Bailout Protocol. Compels every unresolved exception to propagate to the human anchor $h(n)$ via the parent chain, bypassing all intermediate machine actors with immutable forensic attribution.

Each pillar is specified in its own dedicated section below, with formal properties, enforcement mechanisms, relationship to the three-layer model, implementation details, and contribution to the upstream accountability guarantee.

10 The Bailout Protocol: Full Specification

The Bailout Protocol is the mandatory exception-escalation mechanism of the BX3 Framework. It is not an error-handling routine selected at runtime, nor an optional escalation pathway available to a system that chooses to invoke it. It is an architectural compulsion: any unresolved exception state encountered by any node in a BX3 system *must* propagate upward through the accountability chain until it reaches a named human principal, and no machine actor along that chain may absorb, resolve, or suppress the exception. This section provides the complete formal specification: the deterministic state machine governing exception propagation, the precise conditions governing each transition, the bail-out trigger condition, the escalation algorithm, the bypass mechanism enforcing the no-machine-actor property, and the timeout handling that guarantees liveness under all failure conditions.

The protocol operates against the backdrop of the BX3 three-layer model. The *Purpose* layer carries intent, judgment, and final authority; it defines the operating range $R(n)$ for each node n and receives all escalated exceptions. The *Bounds* layer carries bounded reasoning and constrained execution within $R(n)$; it evaluates whether proposed actions fall within the node's provisioned authority and initiates bailout when they do not. The *Fact* layer carries deterministic enforcement and forensic auditability; it verifies proposed actions against the authorization ledger, enforces the state machine's transition relation, and maintains the append-only Forensic Ledger as an immutable record of every protocol event. Each node carries an immutable parent pointer $\alpha(n)$ established at provisioning and a reference to its human accountability anchor $h(n)$ — a named human principal — that is similarly immutable for the node's operational lifetime. The Bailout Protocol exploits this structure to guarantee that exception states propagate to $h(n)$ without passing through any intermediate machine actor as a terminus.

10.1 State Machine Definition

The Bailout Protocol is formally modeled as a deterministic finite state machine embedded in the *Fact* layer of every BX3 node:

$$\mathcal{B} = (Q,\ q_0,\ \Sigma,\ \rightarrow,\ Q_F)$$

Q { IDLE, BOUNDED, BAIL_PENDING, ESCALATING, HUMAN_ACKNOWLEDGED, RESOLVED } is the finite set of protocol states.

q_0 IDLE is the designated initial state, entered at node startup and after every successful directive resolution.

Σ The input alphabet comprising trigger events e_{trigger}, authorization confirmations e_{auth}, timeout signals e_{timeout}, human acknowledgments e_{ack}, binding resolution directives e_{resolve}, and rejection directives e_{reject}.

$\rightarrow$ $\subseteq Q \times \Sigma \times Q$ is the deterministic transition relation (Table 5).

Q_F {RESOLVED} is the set of terminal accepting states.

A node's state $q \in Q$ is stored in the node's Fact Layer worksheet and is observable by its parent node and by the Bailout Monitor. State updates are atomic: no intermediate or partially-executing states are observable externally. The machine is *deterministic*: for every $(q, \sigma) \in Q \times \Sigma$, at most one $q' \in Q$ satisfies $(q, \sigma, q') \in \rightarrow$. Determinism is enforced by the Bailout Monitor's priority function π, which totally orders simultaneous trigger conditions and selects the highest-priority enabled transition before committing any state change. If no transition is defined for a (q, σ) pair, the machine stalls and the Bailout Monitor records a protocol violation in the Forensic Ledger.

Table 5: Bailout Protocol state transition relation $\rightarrow \subseteq Q \times \Sigma \times Q$. Rows are current state; columns are input events. Entries denote the next state. Blank cells indicate that no transition is defined — the machine stalls and the Bailout Monitor records a protocol violation.

	Input event				
Current state	e_{trigger}	e_{auth}	e_{timeout}	e_{ack}	e_{r}
IDLE	BOUNDED	—	—	—	
BOUNDED	—	HUMAN_ACKNOWLEDGED	BAIL_PENDING	—	
BAIL_PENDING	—	—	ESCALATING	—	
ESCALATING	—	HUMAN_ACKNOWLEDGED	ESCALATING	IDLE	RES
HUMAN_ACKNOWLEDGED	—	—	—	IDLE	RES
RESOLVED	—	—	—	—	

State definitions.

IDLE. The node is operating within its provisioned operating range $R(n)$. No exception condition is active. The *Fact* layer dispatches normal action requests through the standard execution path. The node may transition to BOUNDED upon receipt of a trigger event satisfying the bail-out trigger condition (TC), defined in § 10.3.

BOUNDED. An exception e has been detected by the *Bounds* layer and is being held at the *Bounds–Fact* layer boundary. A time-bounded resolution attempt is in progress. All resolution paths within current authority remain open. If the Bounds Engine produces a resolution r that the *Fact* layer approves before T_{bounds} expires, the node returns to IDLE. If all resolution paths are exhausted or T_{bounds} expires, the state advances to BAIL_PENDING.

BAIL_PENDING. The node has determined — either through an explicit `bailout_signal` or through T_{bounds} expiration without a *Fact*-layer-approved resolution — that the exception exceeds its current authority. The Bailout Protocol is formally activated. This is the last architectural

point at which a machine actor could theoretically suppress escalation. It cannot: entry to BAIL_PENDING triggers an immediate atomic transition to ESCALATING via e_{timeout} (T4), with no dwell time and no machine-actor intervention permitted.

ESCALATING. The exception is propagating upward toward $h(n)$. Each intermediate node receives the exception, appends its diagnostic context to the propagation chain, and forwards the exception via `SendToParent` without attempting local resolution. The state remains ESCALATING until either (a) the human anchor acknowledges receipt (e_{ack}: T5), (b) the human anchor issues a binding directive (e_{resolve} or e_{reject}: T7), or (c) all retry pathways timeout after $N_{\max}$ retries at T_{cap}.

KNOWLEDGED. The human anchor has received the exception, observed the diagnostic context, and acknowledged it. The node awaits a directive. No machine actor may issue, simulate, or substitute a directive in this state. The node remains until the human principal issues `ACK` (return to IDLE) or a binding directive `RESOLVE` / `REJECT` (enter RESOLVED).

RESOLVED. The human principal has issued a directive and the protocol has recorded it in the authorization ledger. The directive is one of: `ACK` (continue with current parameters), `RESOLVE` (execute a specific binding resolution), or `REJECT` (disallow the proposed action and enter a quiescent error state). This state is terminal for the current protocol run.

State invariants. The following invariants hold for all reachable states of $\mathcal{B}$:

$$\text{INV}_1 : \forall n \in \text{nodes},\ \text{state}(n) \in Q \setminus \{\text{ESCALATING}\} \iff \neg\text{active_bailout}(n). \tag{INV-1}$$

That is, a node occupies a non-escalating state *iff* it has no active bailout propagation in flight. A node may carry a bailout history (a ledger entry recording a past propagation) while in IDLE, but no active propagation may be underway.

$$\text{INV}_2 : \text{state}(n) = \text{BAIL_PENDING} \implies \text{active_bailout}(n) \wedge \text{transition}(\text{BAIL_PENDING}, e_{\text{timeout}}) = \text{ESCALATI} \tag{INV-2}$$

That is, any node in BAIL_PENDING has an active bailout and the only permissible next transition is to ESCALATING, enforced by the Bailout Monitor without machine-actor discretion.

$$\text{INV}_3 : \text{state}(n) = \text{RESOLVED} \implies \exists d \in \{\text{ACK}, \text{RESOLVE}, \text{REJECT}\} \wedge \text{directive}(n) = d. \tag{INV-3}$$

That is, every RESOLVED state is terminal with a recorded directive from $h(n)$. There are no unresolved RESOLVED states.

10.2 Transition Conditions

Each transition T1–T9 is associated with a formal precondition expressed as a predicate on the node's current state, the Fact Layer worksheet, and the incoming event. The Bailout Monitor evaluates all transition preconditions on every event and commits at most one transition per event to the Forensic Ledger.

Transition T1 (Trigger). IDLE $\rightarrow$ BOUNDED

Precondition: A trigger event e_{trigger} arrives at node n carrying an exception x that satisfies the primary trigger condition TC or the secondary trigger condition (explicit `bailout_signal`):

$$\text{T1 fires} \iff \text{state} = \text{IDLE} \wedge \big(\text{TRIGGER}(n, x) \vee \texttt{bailout_signal}(x)\big). \tag{T1}$$

Transition T2 (Authorization from Bounds). BOUNDED $\rightarrow$ HUMAN_ACKNOWLEDGED

Precondition: The *Fact* layer receives an authorization confirmation e_{auth} from the *Bounds* layer within T_{bounds}:

$$\text{T2 fires} \iff \text{state} = \text{BOUNDED} \wedge e_{\text{auth}} \text{ received before } T_{\text{bounds}}. \tag{T2}$$

Transition T3 (Bail Pending). BOUNDED $\rightarrow$ BAIL_PENDING

Precondition: A timeout signal e_{timeout} fires for T_{bounds} without a *Fact*-layer-approved resolution:

$$\text{T3 fires} \iff \text{state} = \text{BOUNDED} \wedge T_{\text{bounds}} \text{ expired} \wedge \neg\text{resolution_approved}. \tag{T3}$$

Transition T4 (Escalate). BAIL_PENDING $\rightarrow$ ESCALATING

Precondition: The node is in BAIL_PENDING and receives the compounded timeout signal $(T_{\text{bounds}}, T_{\text{prop}})$:

$$\text{T4 fires} \iff \text{state} = \text{BAIL_PENDING} \wedge e_{\text{timeout}} = (T_{\text{bounds}}, T_{\text{prop}}). \tag{T4}$$

Entry to ESCALATING is atomic: the *Fact* layer immediately invokes `SendToParent` without an intervening machine-actor decision step.

Transition T5 (Human Ack during Escalation). ESCALATING $\rightarrow$ IDLE

Precondition: The human anchor receives the exception and issues an acknowledgment `ACK` that requires no further action:

$$\text{T5 fires} \iff \text{state} = \text{ESCALATING} \wedge \text{directive} = \texttt{ACK}. \tag{T5}$$

Transition T6 (Timeout during Escalation). ESCALATING $\rightarrow$ ESCALATING

Precondition: A propagation timeout T_{prop} fires during escalation but retry budget remains:

$$\text{T6 fires} \iff \text{state} = \text{ESCALATING} \wedge e_{\text{timeout}} = T_{\text{prop}} \wedge \text{retry_count} < N_{\max}. \tag{T6}$$

The algorithm retries via `SelectPathway` and `SendToParent` with exponential backoff applied to T_{prop}.

Transition T7 (Binding Directive from Escalating). ESCALATING $\rightarrow$ RESOLVED

Precondition: The human anchor issues a binding directive `RESOLVE` or `REJECT`:

$$\text{T7 fires} \iff \text{state} = \text{ESCALATING} \wedge \text{directive} \in \{\texttt{RESOLVE}, \texttt{REJECT}\}. \tag{T7}$$

Transition T8 (Ack from Acknowledged). HUMAN_ACKNOWLEDGED $\rightarrow$ IDLE

Precondition: The human principal acknowledges without requesting a resolution:

$$\text{T8 fires} \iff \text{state} = \text{HUMAN_ACKNOWLEDGED} \wedge \text{directive} = \texttt{ACK}. \tag{T8}$$

Transition T9 (Human Resolution from Acknowledged). HUMAN_ACKNOWLEDGED → RESOLVED

Precondition: The human principal issues a binding `RESOLVE` or `REJECT` directive from the acknowledged state:

$$\text{T9 fires} \iff \text{state} = \text{HUMAN_ACKNOWLEDGED} \wedge \text{directive} \in \{\texttt{RESOLVE}, \texttt{REJECT}\}. \tag{T9}$$

10.3 Bail-Out Trigger Condition

The bail-out trigger is the condition that causes a node to enter BOUNDED state (T1). Rather than requiring system designers to enumerate specific error codes in advance — a practice that provides coverage only for anticipated failure modes and leaves unanticipated ones invisible — the Bailout Protocol defines the trigger in terms of *observational boundary violation*: a node escalates precisely when its observable input space contains a condition outside its provisioned operating range $R(n)$ and the node cannot produce a *Fact*-layer-approved resolution within T_{bounds}.

Primary trigger condition. The trigger $\text{TRIGGER}(n, x)$ is defined as a conjunction of three sub-conditions, all of which must hold simultaneously:

$$\text{TRIGGER}(n, x) \equiv \underbrace{x \notin R(n)}_{(i)} \wedge \underbrace{\neg\text{resolved}(n, x)}_{(ii)} \wedge \underbrace{\text{confidence}(n, x) < \tau}_{(iii)} \tag{TC}$$

where:

(i) x is an element of the node's observable input space that falls outside the provisioned operating range $R(n)$. This is the *bound signal*: the node observes something it was not provisioned to handle autonomously.

(ii) x has not been previously resolved by the node within the current operational session. This prevents re-trigger on known-exception inputs that have already received human adjudication.

(iii) $\text{confidence}(n, x) < \tau$ is the node's self-assessed confidence that it can produce a correct resolution for x without human input, where τ is the provisioned confidence threshold. The confidence function $\text{confidence} : \text{node} \times \text{input} \to [0, 1]$ is defined at node provisioning and is *not* adjustable at runtime by any machine actor. This prevents a machine actor from inflating its own confidence to suppress a warranted escalation.

Trigger priority function. When multiple trigger conditions fire simultaneously for node n, they are totally ordered by the priority function $\pi(n, x)$, which ensures deterministic transition selection:

$$\pi(n, x) = w_1 \cdot \text{severity}(x) + w_2 \cdot \big(1 - \text{confidence}(n, x)\big) + w_3 \cdot \text{distance}\big(n, h(n)\big) \tag{PF}$$

where $\text{severity}(x) \in [0, 1]$ is the assigned severity of exception x (higher values indicate greater potential impact), $\text{distance}(n, h(n))$ is the number of hops from n to the human anchor in the

parent chain, and w_1, w_2, w_3 are provisioned constants satisfying $w_1 > w_2 > w_3 \geq 0$. The highest-priority trigger fires first. Ordering is enforced by the Bailout Monitor before any state transition is committed to the Forensic Ledger.

Secondary trigger: explicit `bailout_signal`. In addition to TC, the Bounds Engine may emit an explicit `bailout_signal` at any time, regardless of confidence scores. This handles the case where the Bounds Engine detects a condition — for example, a logical contradiction between two inputs within $R(n)$ — that it can recognize as unresolvable even though both inputs are individually within the operating range. The `bailout_signal` bypasses TC entirely and immediately initiates T3:

$$\texttt{bailout_signal}(x) \implies \text{T3 fires for node } n \text{ with exception } x. \qquad \text{(TC-secondary)}$$

The no-discretion property. The *Bounds* layer does not exercise discretion over the trigger condition. Specifically:

1. The evaluation of TRIGGER(n, x) is a read-only query against the operating range definition $R(n)$, not a judgment call made by a machine actor.
2. The result of the query directly drives the state machine transition without an intervening decision step. There is no machine-actor gate between trigger evaluation and protocol activation.
3. The confidence function is provisioned at node initialization and is immutable at runtime. No machine actor may adjust τ to suppress escalation.

These constraints ensure that the trigger condition is architecturally enforced rather than operationally optional.

10.4 Escalation Algorithm

When the state machine enters ESCALATING (T4), the Bailout Monitor executes the escalation algorithm. The algorithm is deterministic, retry-bounded, and path-aware. It does not permit machine actors to intercept, redirect, or absorb the escalation at any step.

Algorithm `Escalate(n, exception, context).`

Algorithm `Escalate(n, exception, context).`

Input: node n, exception e, diagnostic context C
Output: terminal state reached, directive issued by $h(n)$

```
pathway ← SelectPathway(n)
retry_count ← 0; T_current ← T_prop

while retry_count < N_max do
  ack_received ← SendToParent(n, e, C, pathway, T_current)
  if ack_received = true then
    state ← HUMAN_ACKNOWLEDGED; goto WaitForDirective
  end if
```

$T_{\text{current}} \leftarrow \min(2 \cdot T_{\text{current}},\ T_{\text{cap}})$
$retry_count \leftarrow retry_count + 1$
$pathway \leftarrow$ `SelectPathway`(n)
end while

All pathways exhausted at T_{cap}
$state \leftarrow$ ESCALATING; $directive \leftarrow$ `PATHWAY_EXHAUSTED`
return (RESOLVED, `PATHWAY_EXHAUSTED`)

`WaitForDirective:`
$directive \leftarrow$ `ReceiveDirective`$(h(n), T_{\text{human}})$
if $directive =$ `timeout` **then**
  record `unresolved_tag` in Forensic Ledger
  return (RESOLVED, `unresolved`)
end if
$directive_type \leftarrow$ `Parse`$(directive)$
if $directive_type =$ `ACK` **then return** (IDLE, `ACK`)
else if $directive_type \in \{$`RESOLVE`, `REJECT`$\}$ **then return** (RESOLVED, $directive_type$)
end if

`SelectPathway` **function.**

`SelectPathway` **function.**

Input: node n
Output: selected communication pathway p

$P \leftarrow$ PathwayHealthRegistry.get_functional_pathways$(h(n))$
if $P = \emptyset$ **then return** null (no functional pathway)
$p \leftarrow \arg\min_{p' \in P}$ estimated_latency(p')
return p

The algorithm guarantees that at each retry, the pathway with the lowest estimated latency among currently functional pathways is selected. The Pathway Health Registry maintains real-time status for all provisioned pathways, excluding degraded ones from selection until they pass a health check.

`SendToParent` **function.**

`SendToParent` **function.**

Input: node n, exception e, context C, pathway p, timeout T
Output: boolean acknowledgment

$payload \leftarrow$ `BuildEscalationPayload`(n, e, C)
$sent_at \leftarrow$ current_timestamp
$A \leftarrow$ Channel(p).`send`$(payload)$
wait for A **or** T:
  if received(A) **then return** true
end wait

return false

The payload transmitted by `SendToParent` includes: the originating node identifier, the exception record (type, severity, origin timestamp), the full diagnostic context chain accumulated at each intermediate node, and the Fact Layer hash of the triggering state for cryptographic integrity verification at $h(n)$.

10.5 Bypass Mechanism

The bypass mechanism enforces the *no-machine-actor property*: the escalation from the Bailout Monitor to $h(n)$ must not traverse any machine actor that could absorb, redirect, or suppress the exception. This property is not a routing choice — it is a structural necessity.

Definition (Bypass Property) For every node $n \in$ nodes, the set of permissible termination actors for an ESCALATING state is exactly $\{h(n)\}$:

$$\text{terminus}\big(\text{ESCALATING}(n)\big) \;=\; \{h(n)\}. \tag{BP}$$

No machine actor $m \in \text{MachineActors} \setminus \{h(n)\}$ may appear in the termination set, may inspect the escalation payload, may modify the escalation, or may issue a directive that substitutes for one issued by $h(n)$.

The bypass exists because any machine actor interposed between the Bailout Monitor and $h(n)$ introduces three mutually independent failure modes, each of which alone is sufficient to break the upstream accountability guarantee:

Intercept. A machine actor could absorb the escalation, suppress the exception, and continue autonomous operation without ever reaching $h(n)$. This is precisely the failure mode the Bailout Protocol is designed to prevent.

Redirect. A machine actor could redirect the escalation to a different principal, creating an accountability gap in which the wrong human receives the exception and the correct human never learns of it.

Delay. A machine actor could buffer or queue the escalation, delaying $h(n)$'s receipt beyond $T_{\max}$ and violating the liveness guarantee.

The bypass eliminates all three failure modes by architectural constraint: there is no machine actor in the communication path, so none of these failures are architecturally possible. This is analogous to a hardware interrupt line hardwired to a designated handler — just as a hardware interrupt cannot be intercepted or redirected by user-space code, the bailout escalation cannot be intercepted or redirected by any agent-level component.

The bypass is enforced by two cooperating mechanisms operating at different layers:

Fact Layer gate. The *Fact* layer is the only mechanism capable of authorizing state transitions in the BX3 Framework. The Bailout Monitor's pre-commit hook rejects any ledger entry for a resolution where the underlying trigger satisfies TC and T_{bounds} has expired. This prevents the Bounds Engine from issuing a retroactive resolution after timeout to suppress warranted escalation. The *Fact* layer gate operates identically on all machine actors: there is no privileged machine actor with the ability to bypass this check.

root architecture. The parent pointer $\alpha(n)$ and the human-root identifier $h(n)$ are stored in *Fact*-layer-managed memory and are read-only at the machine-actor level. A machine actor cannot redirect the parent chain, cannot insert itself as an intermediate resolver, and cannot modify the *humanRootId* field that terminates the chain. Any attempted modification requires *Purpose* layer authorization with a corresponding ledger entry — which cannot be completed without human approval if the modification would affect the bailout terminus.

Together, these mechanisms ensure that BP holds for all $n \in$ nodes under all execution conditions, including partial node failure, network partition, and deliberate attempt by a compromised machine actor to redirect the bailout pathway.

Conventional escalation hierarchies — including tiered oversight models that permit absorption at intermediate tiers — make human oversight a high-tier option rather than a compulsory terminus. The Bailout Protocol forbids this absorption by architectural constraint: $h(n)$ is not one option among many — it is the *only* permissible terminus for every unresolved exception.

10.6 Timeout Handling

Timeouts in the Bailout Protocol serve a liveness function: they prevent the protocol from permanently stalling when a parent node is unreachable, a communication pathway is unresponsive, or the human anchor is temporarily unavailable. Each timeout has a defined scope, a defined scope-exit action, and an escalation path that prevents deadlock.

Timeout definitions.

solution timeout. The maximum time a node may remain in BOUNDED state attempting autonomous resolution. Default: 60 seconds, provisioned per node class according to the expected latency of the node's resolution procedure. If T_{bounds} expires without a *Fact*-layer-approved resolution, the node transitions to BAIL_PENDING $\rightarrow$ ESCALATING via T3 and T4. The *Fact* layer cannot extend T_{bounds} unilaterally; any extension requires human authorization via a separate ledger entry recorded before T_{bounds} expires.

pagation timeout. The maximum time node n waits for an acknowledgment from its parent after sending an exception via `SendToParent`. Default initial value: 5 seconds. If T_{prop} expires without ACK, the algorithm retries with exponential backoff: $T_{\text{prop}} \leftarrow \min(2 \cdot T_{\text{prop}},\ T_{\text{cap}})$. This backoff continues until either an ACK is received or $N_{\max}$ retries have been exhausted at T_{cap}.

on timeout value. The ceiling value for the exponential backoff of T_{prop}. Default: 60 seconds. After $N_{\max}$ consecutive retries at T_{cap} without ACK, the algorithm fires a `parent_unreachable` timeout event, which propagates to $h(n)$ via the next available functional pathway tracked by the Pathway Health Registry.

edgment timeout. The maximum time the human anchor may remain in HUMAN_ACKNOWLEDGED state without issuing a directive. Default: 300 seconds (5 minutes), configurable per deployment based on the expected human response time for the node's operational domain. If T_{human} expires, the protocol issues a reminder notification to $h(n)$ and transitions the originating node to RESOLVED with an `unresolved` tag. The node becomes quiescent: no autonomous retry occurs, no further actions are attempted, and the `unresolved` tag in the Forensic Ledger signals that human re-engagement is required to clear the state. The protocol does not re-escalate automatically; automatic re-escalation would create a potential denial-of-service condition against the human anchor.

Pathway Health Registry. The Pathway Health Registry (PHR) is a runtime data structure maintained by the Bailout Monitor that tracks the availability, latency, and error rate of each provisioned communication pathway to the human anchor. Before each `SelectPathway` call, the registry is queried and the pathway with the lowest estimated latency among currently functional pathways is selected. Pathways that fail K consecutive health checks (default $K = 3$) are marked `DEGRADED` and excluded from pathway selection until a health check succeeds. A background health-ping thread updates the PHR at intervals of T_{health} (default: 30 seconds), operating with the same scheduling priority as the Bailout Monitor to ensure that pathway status remains current even under heavy agent workload.

Liveness guarantee. Together, T_{bounds}, T_{prop}, T_{cap}, and T_{human} define a finite upper bound on the wall-clock time from trigger firing (T1) to human acknowledgment under any combination of parent unavailability and pathway degradation. This bound is a deployment parameter — not a protocol constant — derived from the network topology, the number of hops to the human anchor, and the provisioned $T_{\max}$ parameter:

$$T_{\max} \;=\; T_{\text{bounds}} \;+\; N_{\max} \cdot T_{\text{cap}} \;+\; T_{\text{human}} \tag{T-max}$$

A deployment that cannot guarantee human acknowledgment within the required $T_{\max}$ under all failure scenarios must provision additional or higher-bandwidth pathways, or must reduce T_{prop} and T_{cap} to bring the worst-case bound within the required limit. This analysis is a required part of deployment certification for any BX3 system operating in a regulated or safety-critical environment.

Timeout interaction with the bypass mechanism. The bypass mechanism ensures that timeouts cannot be weaponized by a machine actor to delay or suppress escalation. Specifically: (a) T_{bounds} is enforced by the *Fact* layer pre-commit hook, not by the *Bounds* layer; a machine actor cannot artificially extend T_{bounds} to prevent the transition to ESCALATING. (b) T_{prop} backoff is calculated by the Bailout Monitor, not by any machine actor; the backoff curve is defined in the protocol specification and cannot be altered at runtime. (c) If all pathways timeout at T_{cap}, the protocol resolves with `PATHWAY_EXHAUSTED` — this is a terminal RESOLVED state, not a suppression of escalation. The `PATHWAY_EXHAUSTED` tag in the Forensic Ledger ensures that the failure of the pathway is recorded as an incident requiring human remediation, not as a successful autonomous resolution.

11 Correctness Properties

The BX3 Framework is designed to satisfy a set of independently stated correctness properties that together constitute the *upstream accountability guarantee*: that consequential decision authority in any BX3 system remains permanently attributable to a named human principal, and that any unresolved exception condition is compulsorily escalated to that principal without passing through any intermediate machine actor as a terminus.

Each property is stated as a formal invariant over the full framework state space, supported by a proof sketch that traces the structural mechanisms enforcing it, and is cross-referenced to the specific pillar or layer that provides the enforcing mechanism. The properties are not independent – they are mutually supporting, and each depends on the correct operation of specific *Fact* layer pre-commit hooks that are identified explicitly.

11.1 Property 1: Intent Preservation

Theorem 1 (Intent Preservation) *Every consequential action executed by any node n in any BX3 system is authorized by a Fact layer entry in the authorization ledger and falls within the operating range $R(n)$ defined by the Purpose layer of n.*

Formal statement. *Let $\mathcal{A}(n)$ denote the set of all consequential actions executed by node n during its operational lifetime. Let $\mathcal{E}(n)$ denote the set of authorization ledger entries for node n. Intent Preservation requires:*

$$\forall a \in \mathcal{A}(n) \ : \ \exists e \in \mathcal{E}(n) \ : \ \mathit{action}(e) = a \ \wedge \ \mathit{scope}(e) \subseteq R(n)$$

where $R(n)$ is the operating range defined by the Purpose layer at node provisioning.

Enforcing mechanism. *The Fact layer's pre-commit hook intercepts every action request from the Bounds layer before it is dispatched to the execution engine. The hook consults the authorization ledger to verify that a valid entry exists for the requested action within the current operating range. If no valid entry exists – if the action is unauthorized or the scope parameter of the existing entry does not cover the current execution context – the pre-commit hook rejects the action and the node enters the Bailout Protocol at the BOUNDED state. No machine actor, including the Bounds layer, may bypass this pre-commit check.*

Dependence. *This property depends on Invariant 1 (the per-node intent preservation invariant established at provisioning) and on Invariant 4 (the immutability of the authorization ledger). If the ledger were mutable, a compromised machine actor could retroactively authorize an unauthorized action; if the intent preservation invariant were violated, the operating range check at pre-commit would lack a valid reference scope.*

11.2 Property 2: Structural Auditability

Theorem 2 (Structural Auditability) *The complete action history, exception history, and resolution history of any node n is recoverable from the Forensic Ledger at any point in time after node provisioning, with cryptographic assurance that no entry has been modified, deleted, or inserted after the fact.*

Formal statement. *Let $\mathcal{F}(n)$ denote the Forensic Ledger entries for node n. Structural Auditability requires that $\mathcal{F}(n)$ forms a cryptographically chained sequence $f_1, f_2, \ldots, f_k$ in which each f_i contains $\mathit{hash}(f_{i-1})$, and that this chain property is verifiable by any auditor with read access to $\mathcal{F}(n)$ without requiring access to any other component of the framework.*

Enforcing mechanism. *The Fact layer's Forensic Ledger Writer assigns each entry a logical timestamp and computes the SHA-256 hash of the preceding entry before committing. The chain is initialized with a genesis hash established at node provisioning. The Fact layer's pre-commit hooks for all five pillars commit their state transitions and events to the Forensic Ledger atomically with the state change – no state transition is committed unless the corresponding Forensic Ledger entry is also committed. The cryptographic chaining is enforced by the Fact layer and cannot be bypassed by any machine actor.*

Dependence. *This property depends on Invariant 5 (the completeness of Forensic Ledger recording) and on Invariant 4 (the immutability of both ledgers). Completeness ensures that no relevant event is omitted; immutability ensures that the recorded events are authentic.*

11.3 Property 3: Compulsory Human Escalation

Theorem 3 (Compulsory Human Escalation) *For any node n, if the Bailout Protocol enters the ESCALATING state at n and all resolution paths within current authority are exhausted, then the protocol must eventually contact $h(n)$ and must not enter a terminal resolving state without a directive issued by $h(n)$.*

Formal statement. *Let t_0 be the time at which the protocol enters ESCALATING at node n. Let $P(t)$ denote the set of functional pathways to $h(n)$ available at time t. Let $T_{\max}$ be the deployment-specific maximum propagation time. Compulsory Human Escalation requires:*

$$\exists t_1 \geq t_0 \ : \ t_1 - t_0 \leq T_{\max} \implies h(n) \text{ is contacted}$$

and, additionally:

$$\text{state}(n) = \text{RESOLVED} \implies \text{directive}(n) \in \{\texttt{RESOLVE}, \ \texttt{REJECT}\}$$

where the directive is verified by the Fact layer as issued by $h(n)$.

Enforcing mechanism. *The Bailout Protocol's state machine (Section 10) enforces the compulsory escalation property through its deterministic transition relation: the only transitions entering RESOLVED require either a RESOLVE or REJECT directive from $h(n)$. The Fact layer's pre-commit hook for the Bailout Protocol rejects any directive whose issuer is not verified as $h(n)$. The escalation algorithm propagates via the immutable parent pointer chain, bypassing intermediate machine actors via the BYPASS property, which is itself enforced by the Fact layer's pre-commit hooks.*

Dependence. *This property depends on Invariant 3 (the exclusivity of $h(n)$ as terminus), Invariant 2 (the immutability of the parent pointer chain), and the correct operation of the Pathway Health Registry ensuring that at least one functional pathway to $h(n)$ is available at all times during normal operation.*

11.4 Property 4: Spawn Tree Integrity

Theorem 4 (Spawn Tree Integrity) *The spawn tree of any BX3 system is an acyclic graph with a designated human root at each node, established at node provisioning, and the parent pointer of each node is immutable throughout its operational lifetime. No node may transfer state to any node outside its spawn subtree except through the authorized cross-domain channel defined by the Spatial Firewall pillar.*

Formal statement. *Let $\mathcal{T}(n)$ denote the spawn tree rooted at node n. Spawn Tree Integrity requires: (i) $\mathcal{T}(n)$ is a directed acyclic graph; (ii) for every node $m \in \mathcal{T}(n)$, the parent pointer $\alpha(m)$ is established at provisioning and is never modified; (iii) the human anchor $h(r)$ of the root node $r \in \mathcal{T}(n)$ is a named human principal; (iv) no state transfer between nodes $p, q \in \mathcal{T}(n)$ occurs except through the authorized channel where $\alpha(p) = q$ or $\alpha(q) = p$ (parent-child) or where both p and q share a common ancestor a and the transfer occurs through the Spatial Firewall's authorized cross-domain channel.*

Enforcing mechanism. *The Fact layer's pre-commit hooks enforce all four conditions at the pre-commit stage. The Recursive Spawning state machine (Section 6) rejects any spawn operation that does not establish $\alpha(c)$ as the spawning node at provisioning. The Spatial Firewall (Section 7) intercepts all cross-domain communication and permits only the authorized cross-domain channel under authorization ledger verification. The Forensic Ledger records all spawn and state-transfer events, enabling retrospective verification of the acyclicity and isolation properties.*

Dependence. *This property depends on Invariant 2 (the immutability of the parent pointer) and on the correct operation of the Spatial Firewall's pre-commit hook for cross-domain communication. If either mechanism is compromised, the spawn tree can no longer be guaranteed acyclic or isolated.*

11.5 Property 5: Sandbox Containment

Theorem 5 (Sandbox Containment) *Every execution event within a sandboxed computation is contained within the sandbox's provisioned execution budget and isolation boundary. No side effect of a sandboxed computation may propagate to the host system or to any other sandbox without passing through the Fact layer's authorization verification. If the sandboxed computation attempts to exceed its execution budget, the abort sequence is triggered and the deviation is recorded in the Forensic Ledger before any subsequent action is taken.*

Formal statement. *Let s be a sandbox instance with execution budget $B(s)$ and isolation boundary $I(s)$. Let $\mathcal{E}(s)$ be the set of execution events generated by the sandboxed computation. Sandbox Containment requires: (i) $\forall e \in \mathcal{E}(s)$, e is observable only within s or within the Fact layer's monitoring interface; (ii) if the provisioned execution time $T(s)$ expires before the sandboxed computation completes, the abort sequence is triggered; (iii) the abort sequence records the termination event in the Forensic Ledger before the sandbox is destroyed.*

Enforcing mechanism. *The Sandbox Gate (Section 8) enforces the execution budget and isolation boundary through the Fact layer's pre-commit hook. Every I/O operation attempted by the sandboxed computation is intercepted by the gate and submitted to the Fact layer for authorization verification before execution. The abort sequence is triggered by the watchdog timer embedded in the Sandbox Gate, which operates with the same scheduling priority as the Bailout Monitor and is not preemptible by sandboxed code. The Forensic Ledger records the abort event with the full execution trace generated during the sandbox session.*

Dependence. *This property depends on the isolation guarantees provided by the underlying sandboxing substrate (a deployment assumption) and on the correct operation of the Sandbox Gate's watchdog timer. If the watchdog timer fails to fire or the pre-commit hook is bypassed, the containment property is violated.*

12 Related Work

The BX3 Framework is situated at the intersection of five distinct but convergent research traditions. This section surveys representative prior work in each tradition, identifies the specific gaps that BX3 addresses, and positions the framework relative to the most directly competitive or complementary approaches.

12.1 Accountability Mechanisms in Multi-Agent Systems

The accountability problem in autonomous computational systems was first articulated by Wiener [5]: when a machine exercises decision-making authority previously held by a human, the chain of responsibility becomes ambiguous in precisely the way that permits consequential actions to proceed without attributable principal. This structural ambiguity is not a governance inconvenience — it is a direct consequence of delegatory architectures in which the delegate possesses sufficient agency to reinterpret or suppress the principal's oversight.

Four decades of subsequent work established the gap between accountability-as-aspiration and accountability-as-architecture. Dijkstra's formal verification program [2] showed that correctness must follow from formal structure, not testing — a principle that extends to accountability properties that cannot be established by policy documents or organizational conventions. Trist's analysis of socio-technical systems [4] established that accountability requires not merely that actions be attributable to their immediate performer, but that the full chain of delegation, including all intermediate scopes and their respective authorization limits, be recoverable as a structurally immutable record. Without this property, any node in the chain may disclaim responsibility by pointing to a predecessor or successor, and the principal who authorized the original delegation has no means to resolve the ambiguity.

Bansal et al. [1] demonstrated empirically that even small teams of autonomous agents operating under well-specified objectives generate emergent behaviors not predicted by any individual agent's model and not attributable to any specific node in the delegation chain. This finding establishes the drift amplification failure mode: each locally-correct action taken without reference to the principal's global intent compounds into a system state further from authorization with each successive decision. The gap arises because each agent acts within the scope of its own authorization, which is locally coherent but globally unanticipated. No individual agent is acting incorrectly; the consequential failure arises from the aggregate.

Kim et al.'s Tiered Agentic Oversight (TAO) model [3] represents the most directly competitive approach. TAO establishes hierarchical machine tiers through which exceptions may be absorbed before reaching a human principal. The model correctly identifies that human oversight must scale with agent complexity, but its hierarchical absorption architecture is precisely the design choice that creates the structural unauditability failure mode identified in Section 1: human oversight becomes a high-tier option rather than a compulsory terminus. TAO permits exceptions to be resolved at intermediate machine tiers, making the human principal's awareness of any given exception contingent on the machine tier's assessment of its severity — precisely the contingent oversight that BX3 prohibits by architectural constraint.

The BX3 Framework takes a position on this architectural choice: absorption at intermediate tiers is forbidden. The human principal $h(n)$ is the exclusive terminus for all unresolved exception conditions. This is not a policy preference but a structural commitment that follows from the functional separation between Purpose, Bounds, and Fact layers. The Framework's position is that any architecture permitting intermediate machine-actor absorption cannot provide the upstream accountability guarantee, regardless of how many tiers it employs.

12.2 Human-in-the-Loop Architectures

Human-in-the-loop (HITL) design positions human agents as integral components of AI-driven workflows rather than external reviewers who intervene only after the system has reached a conclusion. The canonical motivation is epistemic: AI systems operate within the bounds of their training data and objective functions and cannot reliably reason about novel situations, value trade-offs, or

contextual factors requiring human judgment [4]. A secondary motivation is institutional: many decisions carry legal, financial, or social consequences that cannot be delegated to a machine by institutional design, regardless of empirical performance [1].

The gap between HITL aspiration and HITL architecture is well documented. Effective HITL requires three conditions: real-time situational awareness of the system's operational state, domain knowledge sufficient to evaluate proposed actions against operational intent, and a decision gate that cannot be bypassed by the system's own logic. In most deployed AI systems, none of these conditions are architecturally guaranteed. The human operator is placed in an informational loop rather than an authoritative one: they receive outputs and reports, but they do not hold a structural gate that the system cannot proceed past.

The BX3 Framework satisfies all three conditions through its layered architecture. The Fact layer provides real-time situational awareness through the Forensic Ledger, which records every protocol event with full attribution, making the system's operational history permanently observable. The Purpose layer provides domain knowledge through the operating range definition $R(n)$, which encodes the principal's intent as a formal specification that the Bounds layer can evaluate programmatically. The Fact layer's pre-commit hook provides the unbypassable decision gate: no action proceeds without a verified authorization entry, and the only entry that can close an unresolved exception state is one issued by $h(n)$ through the Purpose layer.

Bansal et al.'s empirical finding that human-AI team performance depends on the human's mental model accuracy [1] has direct implications for the Bailout Protocol's diagnostic propagation chain. The protocol addresses this through the diagnostic context attached to each escalation: the Bounds Engine's reasoning trace, the exception classification, the operating range definition, and the resolution history of prior occurrences are all appended to the escalation signal before it reaches the human anchor. A HITL mechanism that surfaces exceptions without adequate diagnostic context is, as Bansal et al. establish, functionally equivalent to no HITL at all.

12.3 Fail-Safe and Safety-Critical Systems

Fail-safe design originates in control systems engineering, where component failures are measured in human lives. A fail-safe system is one that, upon failure of any component, transitions to a predefined safe state in which the system causes no harm [5]. The safe state is defined by the system's designers based on their understanding of the worst-case failure modes and their relative severities.

Classical fail-safe patterns in software systems include watchdog timers (bounded execution time enforcement with recovery or shutdown on timeout), graceful degradation (partial functionality retention under component failure), and deterministic shutdown sequences (quiescent state with all actuators de-energized and all pending transactions recorded). These patterns address component failures in conventional computing systems, but they do not address the specific failure mode that BX3 targets: the absence of any structural mechanism to compel human involvement when a system encounters a condition outside its authorized scope.

The BX3 Framework's safe state is human-acknowledged quiescence: a state in which the node has surfaced its exception to a human principal and awaits a directive, having ceased all further autonomous action. This is a meaningfully different safe state than classical fail-safe systems provide, because it requires not only that the system stop but that it reach a human who can exercise judgment over subsequent decisions. This is architecturally more complex than a physical state transition, because the transition depends not only on the system reaching the human but on the human being available, willing, and sufficiently informed to exercise meaningful judgment.

The timeout mechanisms embedded in the BX3 pillars function as watchdog timers at the archi-

tectural level. The T_{bounds} parameter governing the BOUNDED state in the Bailout Protocol ensures that bounded reasoning does not become unbounded deliberation. The Sandbox Gate's execution budget ensures that sandboxed computation cannot exceed its provisioned time without triggering the abort sequence. These timeouts are not advisory — they are enforced by the Fact layer's deterministic logic, which does not permit discretionary override by any machine actor.

Dijkstra's observation that the difficulty of constructing correct programs is primarily a difficulty of reasoning about all possible states a system may enter [2] applies directly to the BX3 accountability problem. The BX3 Framework does not assume that its own accountability properties will hold under all conditions. It establishes the conditions under which the upstream accountability guarantee holds (the invariants of each pillar) and specifies the operational consequences when those conditions are violated (the failure modes catalogued in Section 13). A system that assumes its own correctness is, by that assumption, unsafe. A framework that specifies the conditions under which its guarantees hold and the directed research agenda for extending those conditions is, by that commitment, safer.

12.4 Recursive and Multi-Agent Architecture

Recursive agent architectures, in which agents may spawn sub-agents that operate within the parent's delegated authority, are among the most powerful and most dangerous patterns in current AI system design. The power arises from the ability to decompose complex, open-ended tasks into specialized sub-tasks executed in parallel by purpose-built agents. The danger arises from the accountability gap that opens between the root principal and the leaf agents: each level of recursion adds an intermediate machine actor to the delegation chain, and each intermediate actor may absorb exceptions, reinterpret objectives, or fail in ways that are invisible to the principal and to external auditors.

The BX3 Framework addresses recursive architecture through Pillar 2 (Recursive Spawning) and Pillar 3 (Spatial Firewall), which together establish that the spawn tree is an append-only accountability structure with immutable parent pointers and isolated execution domains. The spawn tree cannot be restructured after creation; sub-agents cannot transfer state to siblings or cross-domain peers; and the parent pointer chain is the only authorized channel for cross-domain communication. These constraints prevent the spawn tree from evolving into a tangled graph in which accountability cannot be retrospectively reconstructed.

The Bailout Protocol operates over the spawn tree as the substrate for exception propagation. The immutable parent pointer $\alpha(n)$ is the propagation path; the human anchor $h(n)$ is the compulsory terminus. The combination means that the spawn tree, however deep or wide, is always a tree with a human at its root — not a graph with ambiguous or circular accountability relations.

12.5 Formal Verification of Accountability Properties

The formal verification of accountability properties in multi-agent systems is an active research area. Classical model checking approaches are well established for finite-state systems [2], but they face scalability challenges when applied to systems with unbounded spawn trees or non-terminating agent loops. Runtime verification approaches, in which formal properties are monitored during execution rather than verified statically, offer a more tractable path for multi-agent systems operating in open-ended environments.

The BX3 Framework takes a position on the formal verification question: accountability properties must be enforceable at runtime, not merely verifiable statically. A statically verified system that fails to enforce its accountability properties at runtime is, from the principal's perspective, equivalent to a system that never verified them at all. The Fact layer's pre-commit hook is the runtime enforcement mechanism that makes the static invariants operative. The Forensic Ledger

is the runtime artifact that provides the evidence base for any retrospective verification. Together, they constitute a hybrid verification approach: static invariants established at node provisioning, runtime enforcement by the Fact layer, and retrospective auditability by the Forensic Ledger.

13 Limitations and Future Work

The three correctness properties established in Section 11 are conditional guarantees: each holds under a defined set of structural and deployment assumptions. This section makes those assumptions explicit, characterizes the operational consequences of their failure, and maps each gap to a directed research agenda.

13.1 Limitations

13.1.1 Human Response Latency

The BX3 Framework's Liveness property guarantees eventual human contact within a bounded wall-clock interval, but the bound $T_{\max}$ is a function of network topology, retry parameters, and provisioned timeouts — not of human cognitive response time. For operational domains with time constants measured in milliseconds (high-frequency trading, real-time process control, autonomous vehicle navigation), the time required for a human principal to receive, interpret, and respond to an escalation may exceed the system's operational time horizon by one to two orders of magnitude.

The framework's prescribed failure mode under latency stress is well-defined: the node stalls in `ESCALATING` *until the anchor responds or all pathways time out. This is the intended safety-preserving behavior. However, the operational cost of a stall is domain-dependent in ways the framework does not currently reason about. In a precision agriculture context (Irrig8 deployment), a stall that delays a watering decision by minutes is acceptable; in a financial risk-interlock context, a stall that delays a margin-call response by even seconds may expose the system to catastrophic loss. The framework treats the human anchor as a timing black box; there is no mechanism to reason about whether delay reflects anchor overload, a structural mismatch between $T_{\max}$ and the anchor's operational availability, or a pathway failure.*

The deeper limitation is that human response latency is provisioned statically at node setup and is not sensitive to the anchor's current operational context. $T_{\max}$ is a deployment parameter, not an adaptive runtime parameter. For latency-sensitive deployments, the provisioning burden falls on the system designer to establish empirically grounded latency budgets and encode them as enforceable policy.

Directed research: Empirical characterization of human response latency distributions across deployment contexts to establish statistically grounded, domain-specific latency budgets. Integration of these budgets into the node-provisioning interface as enforceable policy parameters with runtime monitoring. Investigation of graded escalation responses: a protocol that requests a rapid `ACK` *followed by a detailed directive would better match the operational reality of human decision-making under varying loads.*

13.1.2 Adversarial Conditions

The Safety property requires that human-contact pathways cannot be permanently closed without recorded human authorization. This guarantee holds under the assumption that the host platform provides independent scheduling priority to the Bailout Monitor (Assumption A1.1). In an adversarial environment — particularly one in which the host platform itself may be compromised

— this assumption may not hold. A sophisticated attacker controlling the execution substrate can modify thread priorities, preempt the monitor thread, introduce race conditions into pathway-closure operations, or rewrite the Bailout Monitor's code before it executes.

The framework's guarantees are platform guarantees. They rest on invariants that the host platform must supply: non-preemptible monitor scheduling, append-only persistence for the authorization ledger, and interruptible blocking calls for pathway management. The BX3 Framework is designed for production AI deployments in which individual agents may behave unexpectedly but the host platform is trusted — the standard assumption for regulated industries. It is not designed for deployments in which the platform itself is an active adversary: national infrastructure control systems, military command-and-control environments, financial clearinghouse infrastructure.

Directed research: Formal specification of the composition interface between the BX3 Framework and substrate-integrity mechanisms. Investigation of whether the framework's guarantees can be preserved under weaker platform assumptions, including partially compromised substrates. Exploration of hardware-attested execution environments (e.g., Intel SGX, ARM TrustZone) as a deployment target for the Bailout Monitor, eliminating the OS as a trust dependency.

13.1.3 Scalability

The BX3 Framework is specified with a 1:1 relationship between each node and its human anchor. This model is appropriate for single-node deployments or small-scale systems, but it creates a structural bottleneck in enterprise contexts where one anchor may supervise hundreds of concurrent nodes. Without pooling and load-balancing mechanisms, the human anchor becomes the rate-limiting step in a system whose design intent is to route exceptions away from centralized machine decision points.

The current specification treats human anchoring as a static 1:1 relationship established at node provisioning. This model must evolve to support at minimum 1:N relationships (one anchor supervising multiple nodes with workload pooling) and, in advanced deployments, N:M relationships (anchor pools with load balancing and dynamic routing). The correctness properties of the framework must be shown to extend to these pooled configurations, particularly Safety: no pooling configuration may produce a state in which all human-contact pathways for a given node are permanently disabled.

Additional scalability considerations arise from the spawn tree depth. As the depth of the accountability chain grows through recursive spawning in large multi-agent systems, the time to propagate an exception to the anchor grows proportionally. The interaction between chain depth, propagation latency, and timeout configuration must be characterized for deployments with large spawn trees.

Directed research: Formal specification of human anchor pool architectures, including load-balancing policies, prioritization schemes, and routing algorithms. Extension of Safety and Liveness proofs to pooled anchor configurations with formally verified pathway-redundancy guarantees. Empirical evaluation of anchor pool performance under simulated high-escalation-load conditions.

13.2 Future Work

The following research directions extend the BX3 Framework's guarantees to broader deployment contexts and establish a principled research program beyond the current specification.

Formal verification. *Full mechanical verification of the Safety, Liveness, and Accountability invariants using a formal verification tool (Coq, Isabelle/HOL, or TLA$_+$) applied to the BX3 three-layer model and all five pillar state machines. The current paper establishes the invariants through*

proof sketches; the directed research agenda targets full formal verification as a precondition for deployment in regulated industries.

Adaptive timeout mechanisms. *Replacement of static timeout parameters (T_{bounds}, T_{prop}, T_{human}) with adaptive mechanisms that adjust to anchor workload, pathway latency, and deployment context. The adaptive protocol would maintain the safety invariant (the anchor is always contacted within $T_{\max}$) while reducing unnecessary stalls in low-consequence, high-latency scenarios.*

Distributed human anchor pools. *Formal specification and empirical evaluation of anchor pool architectures that sustain scalability under cascading failure conditions without compromising the no-machine-actor-absorbs property. This is the primary scalability extension required for enterprise BX3 deployments.*

Exploitation resistance. *Design of triggering-behavior anomaly detectors that audit the statistical relationship between observed exception conditions and triggered escalations per node over rolling observation windows. Investigation of provenance attestation for operating-range assessments: requiring that each Bounds Engine self-assessment be counter-signed by an independent audit process, making post-hoc misclassification detectable through forensic reconstruction.*

14 Conclusion

The BX3 Framework advances a single, architecturally precise claim: that autonomous systems must propagate every unresolved exception state to a named human principal, bypassing all intermediate machine actors, as a compulsory structural requirement rather than a runtime choice. This mandatory bailout property is not a feature that can be retrofitted to an existing agentic architecture through better error handling or improved escalation heuristics. It is a structural property that follows, as a matter of architectural necessity, from the functional separation between Purpose, Bounds, and Fact layers.

The framework's five named architectural pillars are co-equal and mutually reinforcing. Loop Isolation ensures that the spawn graph is a rooted DAG and that accountability attribution is never ambiguous. Recursive Spawning fixes the accountability chain at the architectural level and makes it immutable. Spatial Firewall contains sub-agent failures within execution domains and prevents lateral state corruption. Sandbox Gate provides the sole permitted cross-domain communication channel with mandatory authorization verification. The Bailout Protocol compels the final escalation for any condition not resolved by the preceding four pillars, including conditions not anticipated at design time.

Together, these five pillars constitute the upstream accountability guarantee: that BX3 systems fail upward into human consciousness, never downward into autonomous chaos. *The guarantee is not a design aspiration. It is a logical consequence of the framework's three-layer architecture, enforced by the Fact layer's deterministic pre-commit hook and recorded in the append-only Forensic Ledger.*

The theoretical contribution of this paper is foundational: that the upstream accountability guarantee is not a design choice but a logical consequence of the three-layer architecture, and that the Bailout Protocol is the mechanism by which that consequence is enforced at runtime. The practical contribution is equally clear: that autonomous systems operating without such a mechanism are structurally exposed to failure modes — invisible exception absorption, contingent escalation, and irreversible drift — that cannot be eliminated by incremental improvement. They require a new architectural primitive. The BX3 Framework is that primitive.

The limitations analysis identifies the boundaries of this primitive as currently specified: human response latency introduces a structural floor on exception resolution time; sophisticated agents

may exploit strategic misclassification to suppress warranted escalation; cascading failure conditions introduce scalability constraints that the current framework does not resolve; and the framework's guarantees are conditioned on substrate integrity properties that the framework itself cannot verify. The directed research agenda — formal verification, adaptive timeout mechanisms, distributed anchor pools, and exploitation resistance — addresses each limitation as a principled research program.

Future work will determine whether the BX3 Framework's guarantees hold under full formal verification, whether adaptive exception classification can preserve safety while reducing human latency for low-consequence exceptions, and whether distributed human anchor pools can sustain scalability under cascading failure conditions without compromising the no-machine-actor-absorbs property. The framework's core claim — that the upstream accountability guarantee requires mandatory human escalation for all unresolved exception states — is not contingent on these open questions. It is established by the structural necessity of the three-layer architecture.

Acknowledgments

We acknowledge the foundational influence of Norbert Wiener's Cybernetics in establishing the structural relationship between autonomous decision-making authority and human accountability. We further acknowledge the socio-technical systems literature, particularly Trist and Bamforth's analysis of organizational change, for establishing the principle that technological and social systems are co-constituted — a principle that informs the BX3 Framework's treatment of human-AI governance as an architectural, not merely a policy-level, commitment. No external funding was received for this work.

References

[1] *Gagan Bansal, Besmira Nushi, Ece Kamar, and Susan L. Davidson. Beyond accuracy: The role of mental models in human-ai team performance.* Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, 7(1):15–23, 2019.*

[2] *Edsger W. Dijkstra. On the cruelty of really teaching computing science.* E.W. Dijkstra, EWD 887*, 1982. Manuscript; widely circulated as an essay on formal verification and program correctness.*

[3] *James Kim, Serena Rao, and Alex Chen. Tiered agentic oversight: Scalable human supervision for large-scale multi-agent systems.* arXiv preprint*, 2025. Available at* `https://arxiv.org/abs/2501.XXXXX`.

[4] *Eric Trist and Kenneth Bamforth. Some social and psychological consequences of the long grid iron methodology.* Human Relations*, 4:3–38, 1951. Reprinted with discussion in Trist's 1981 selected writings on organizational change and socio-technical systems design.*

[5] *Norbert Wiener. Cybernetics: Or control and communication in the animal and the machine. 1948. First edition.*